BCSE305L-EMBEDDED SYSTEMS

MODULE-2

I/O Interfacing Techniques

MODULE-2

I/O Interfacing Techniques

- Memory interfacing.
- A/D, D/A.
- Timers.
- Watch-dog timer.
- Counters.
- Encoder & Decoder.
- UART.
- Sensors and actuators interfacing.

INTRODUCTION TO ARDUINO UNO

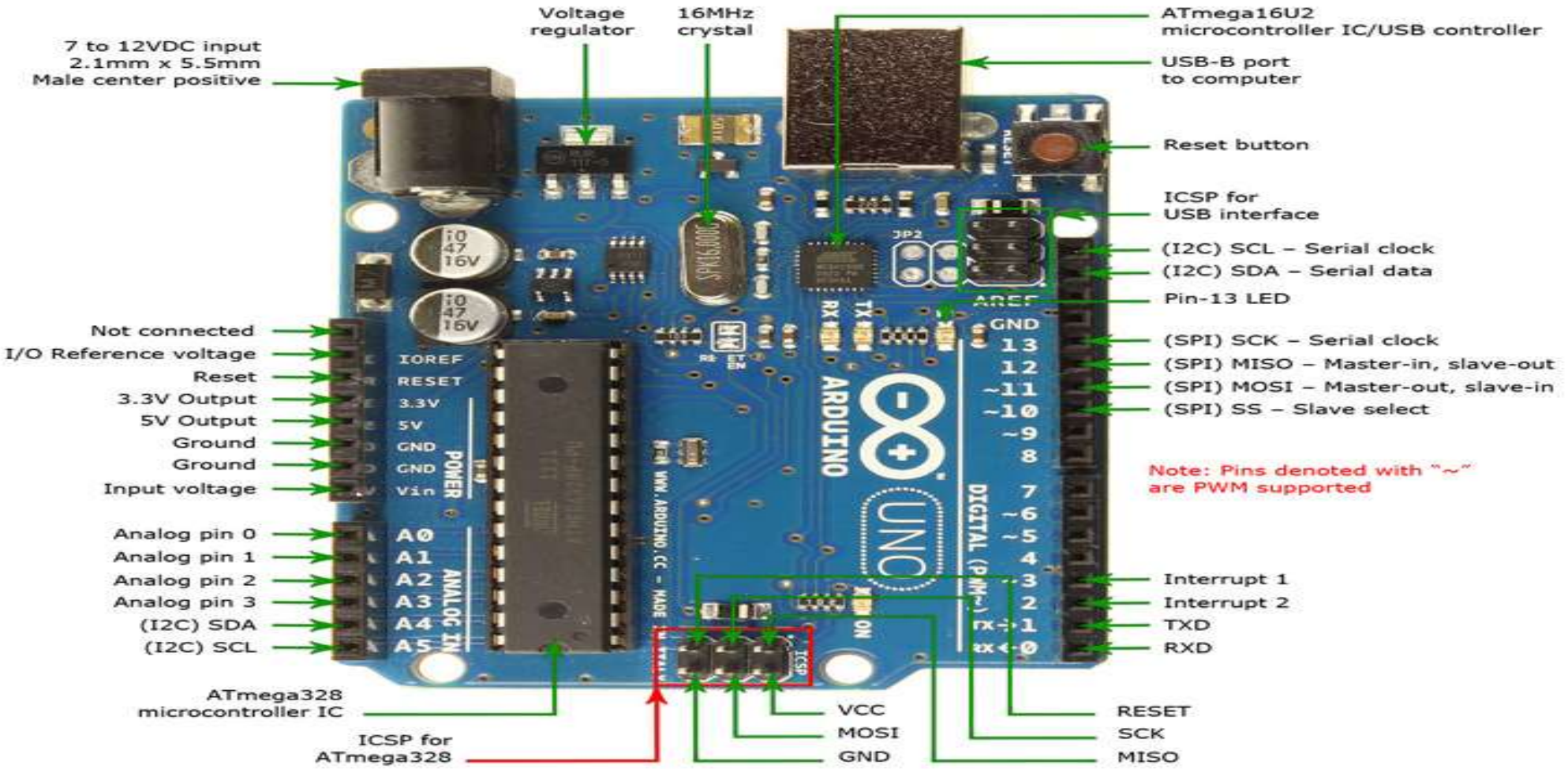
INTRODUCTION TO ARDUINO UNO

Features

- An open-source microcontroller board
- Microcontroller : ATmega328
- Operating Voltage : 5V
- Input Voltage (external) : 6-20V (7-12 recommended)
- Digital I/O Pins : 14 (6 PWM output)
- Analog Input Pins : 6
- DC Current per I/O Pin : 40 mA
- DC Current for 3.3V Pin : 50 mA
- Flash Memory : 32 KB (0.5 KB for bootloader)
- Clock Frequency : 16 MHz
- Programming Interface : USB (ICSP)

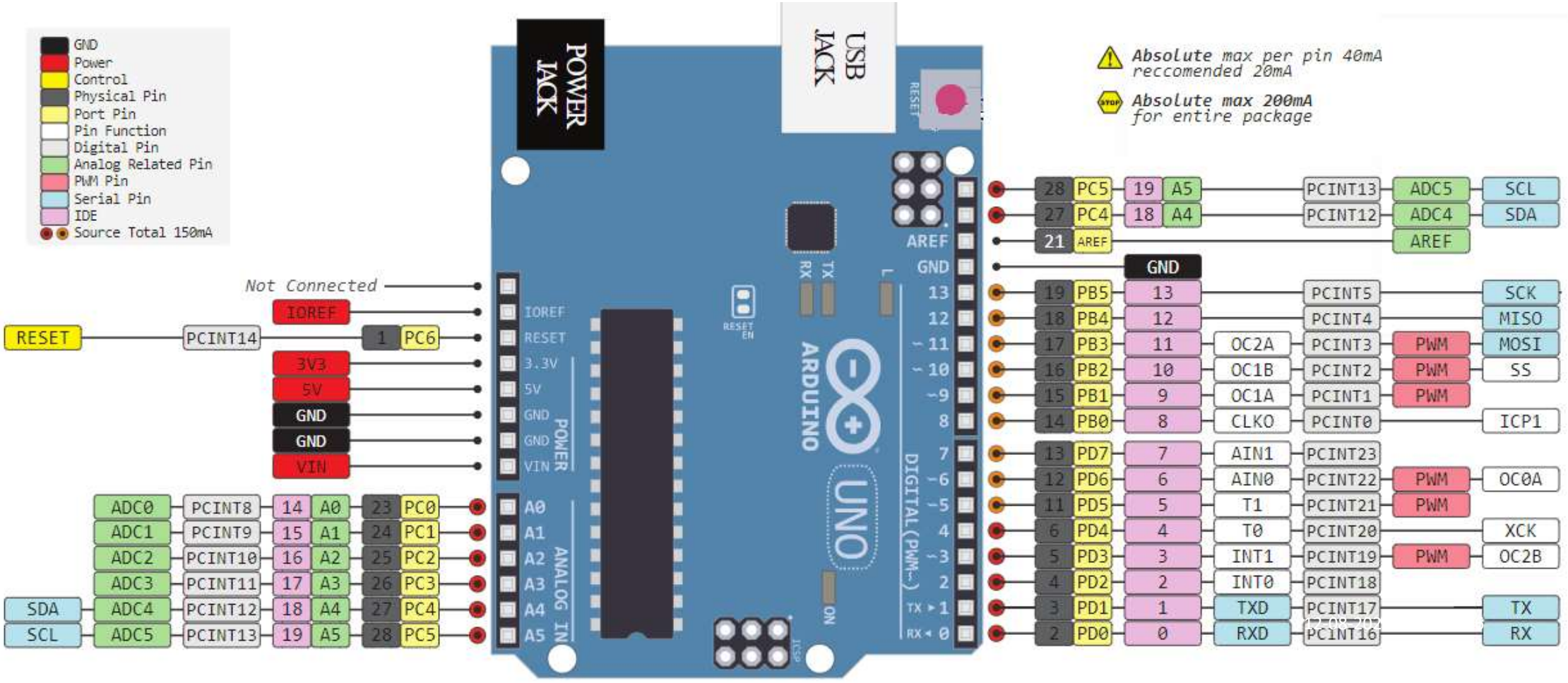
INTRODUCTION TO ARDUINO UNO

Arduino Uno - Board Details



INTRODUCTION TO ARDUINO UNO

Arduino Uno - Pin Details

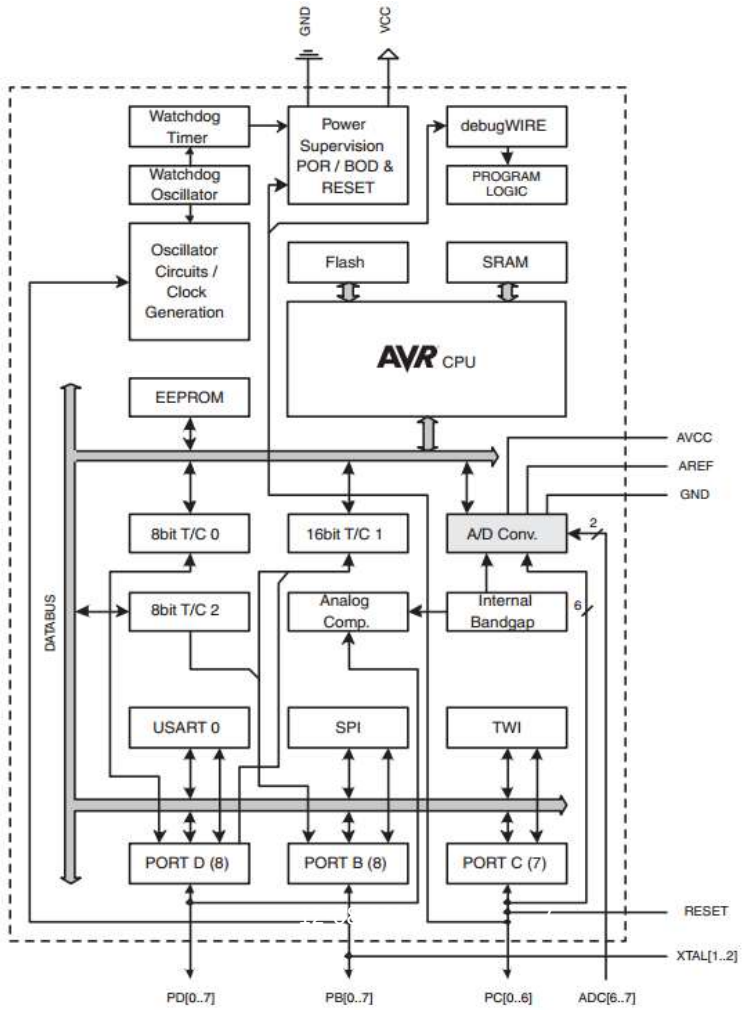


INTRODUCTION TO ATmega328

Features

- Instruction set Architecture : RISC
- Data bus size : 8-bit
- Address bus size : 16-bit
- Program memory (ROM) : 32 KB
- Data memory (RAM) : 2 KB
- Data memory (EEPROM) : 1 KB
- Input/output ports : 3 (B, C, D)
- General purpose registers : 32
- Timers : 3 (Timer0, 1 & 2)
- Interrupts : 26 (2 Ext. interrupts)
- ADC modules (10-bit) : 6 channels
- PWM modules : 6
- Analog comparators : 1
- Serial communication : SPI, USART, TWI(I2C)

Architecture



PROGRAMMING I/P and O/P

- Information exchange is done by input and output devices
- It may be digital or analog signal
- Digital signals will be directly interfaced to CPU
- Analog devices requires some converter for interfacing with CPU
- All peripherals can not be always on-chip

PROGRAMMING I/P and O/P

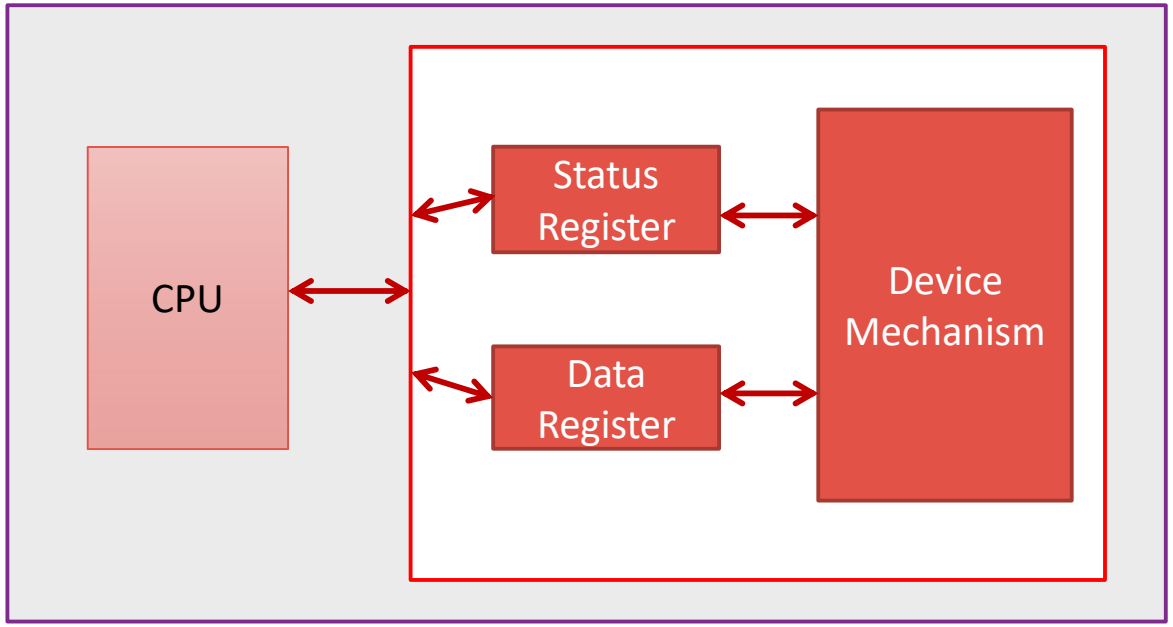
- Internal I/O devices
  - Physically silicon devices
  - Small size

  Ex: RTC, ADCs, memory
- External I/O devices
  - Have large size
  - Has bus interface

  Ex: LCD, keypad, MIC

PROGRAMMING INPUT & OUTPUT

- CPU communicates to the device by reading and writing to the registers.
- Data register – read and write, Status register – read only



PROGRAMMING INPUT & OUTPUT

- Microcontroller programming input and output
  - I/O mapped I/O
  - Memory mapped I/O (widely used)

- Memory mapped I/O –
  - Common address shared by both memory and I/O
  - Read and write instructions to communicate with the devices
  - Example: In 8051 Timer Register - MOV TMOD,#01H

PROGRAMMING INPUT & OUTPUT

- **Busy-Wait I/O**
  - Checking an I/O device if it has completed the task by reading its status register often called polling.
  - Example: AGAIN: JNB TF0, AGAIN
- **Interrupts**
  - It enables I/O devices to signal the completion or status and force execution of a particular piece of code.
  - Example: MOV IE, #82H

PROGRAMMING ADC & DAC

ANALOG TO DIGITAL (ADC) CONVERTER

- An ADC is an electronic circuit whose digital output is proportional to its analog input.
- Effectively it "measures" the input voltage and gives a binary output number proportional to its size.
- The list of possible analog input signals is endless, (e.g. audio and video, medical or climatic variables)
- The ADC when operates within a larger environment, often called a data acquisition system.

ANALOG TO DIGITAL (ADC) CONVERTER

- ADC conversion

$$D = \frac{V_i}{V_r} \times 2^n$$

Where,

- V_i is the input voltage
- V_r the reference voltage
- n the number of bits in the converter output
- D the digital output value.

- The output binary number D is an integer
- For an n-bit number can take any value from 0 to $(2^n - 1)$.

ANALOG TO DIGITAL (ADC) CONVERTER

- ADC has maximum and minimum permissible input values.
- Resolution is a measure of how precisely an ADC can convert and represent a given input voltage.

$$Resolution = \frac{V_r}{2^n}$$

- Arduino ADC is 10-bit. This leads to a resolution of 5/1024, or 4.88 mV.

ANALOG TO DIGITAL (ADC) CONVERTER

- Not all the pins on a microcontroller have the ability to do analog to digital conversions.
- In Arduino board, analog pins have an ‘A’ in front of their label (A0 through A5).
- A 10-bit device a total of 1024 different values.
- An input of 0 volts would result in a decimal 0.
- An input of 5 volts would give the maximum of 1023.

ANALOG TO DIGITAL (ADC) CONVERTER

- ADC value to voltage conversion
- ADC value – 434

$$434 = \frac{V_i}{5} \times 1024$$

$$V_i = 2.12\ \text{V}$$

ANALOG TO DIGITAL (ADC) CONVERTER

Analog I/O

- AREF pin and analogReference(type) ADC reference other than 5 V.
- Increase the resolution available to analog inputs
  - That operate at some other range of lower voltages below +5
  - Must be declared before analogRead()

analogReference(type)

Parameters type:
which type of reference to use (DEFAULT, INTERNAL, INTERNAL1V1, INTERNAL2V56, or EXTERNAL)
Returns
None

ANALOG TO DIGITAL (ADC) CONVERTER

Analog I/O

analogRead(pin)

- Reads the value from a specified analog pin with a 10-bit resolution.
- Works for analog pins (0–5).
- It will return a value between 0 to 1023.
- 100 microseconds for one reading.
- Reading rate 10000 per sec.
- INPUT nor OUTPUT need not be declared.

analogRead(pin)

Parameters pin:
the number of the analog input pin to read from (0-5)

Returns
int(0 to 1023)

ANALOG TO DIGITAL (ADC) CONVERTER

Analog I/O
analogWrite(pin,value)

- Write an analog output on a digital pin.
- This is called Pulse Width Modulation.
- PWM uses digital signals to control analog devices.
- Digital Pins # 3, # 5, #6, # 9, # 10, and # 11 can be used as PWM pins.
- No need to configure pin as an OUTPUT.

analogWrite(pin,value)

Parameters pin: the number of the pin you want to write
value: the duty cycle between 0 (always off, 0%) and 255 (always on, 100%)

Returns
None

ANALOG TO DIGITAL (ADC) CONVERTER

EXAMPLE 1

Program an Arduino board to read analog value from an analogy input pin, convert the value into digital value and display it in a serial window.

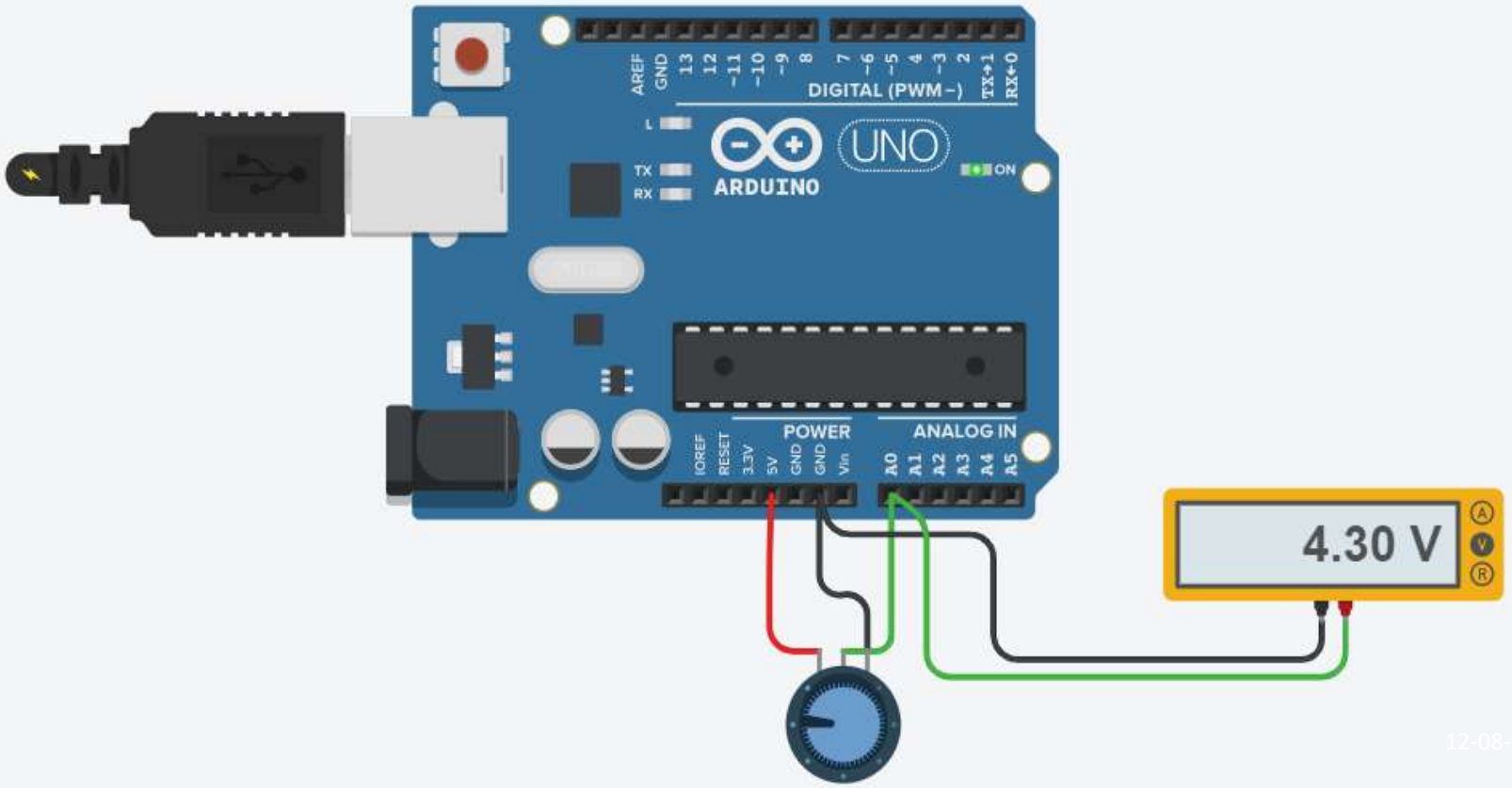
ANALOG TO DIGITAL (ADC) CONVERTER

CODE

```
int sensorValue = 0;

void setup()
{
//monitor output in serial port
Serial.begin(9600); //intitating serial communication with 9600 baud rate
}

void loop()
{
  sensorValue = analogRead(A0);
  //analog input is read from A0 pin
  Serial.println(sensorValue);
  //printing the output in serial port
  delay(100); // simple delay of 100ms
}
```

ANALOG TO DIGITAL (ADC) CONVERTER

CODE



ANALOG TO DIGITAL (ADC) CONVERTER

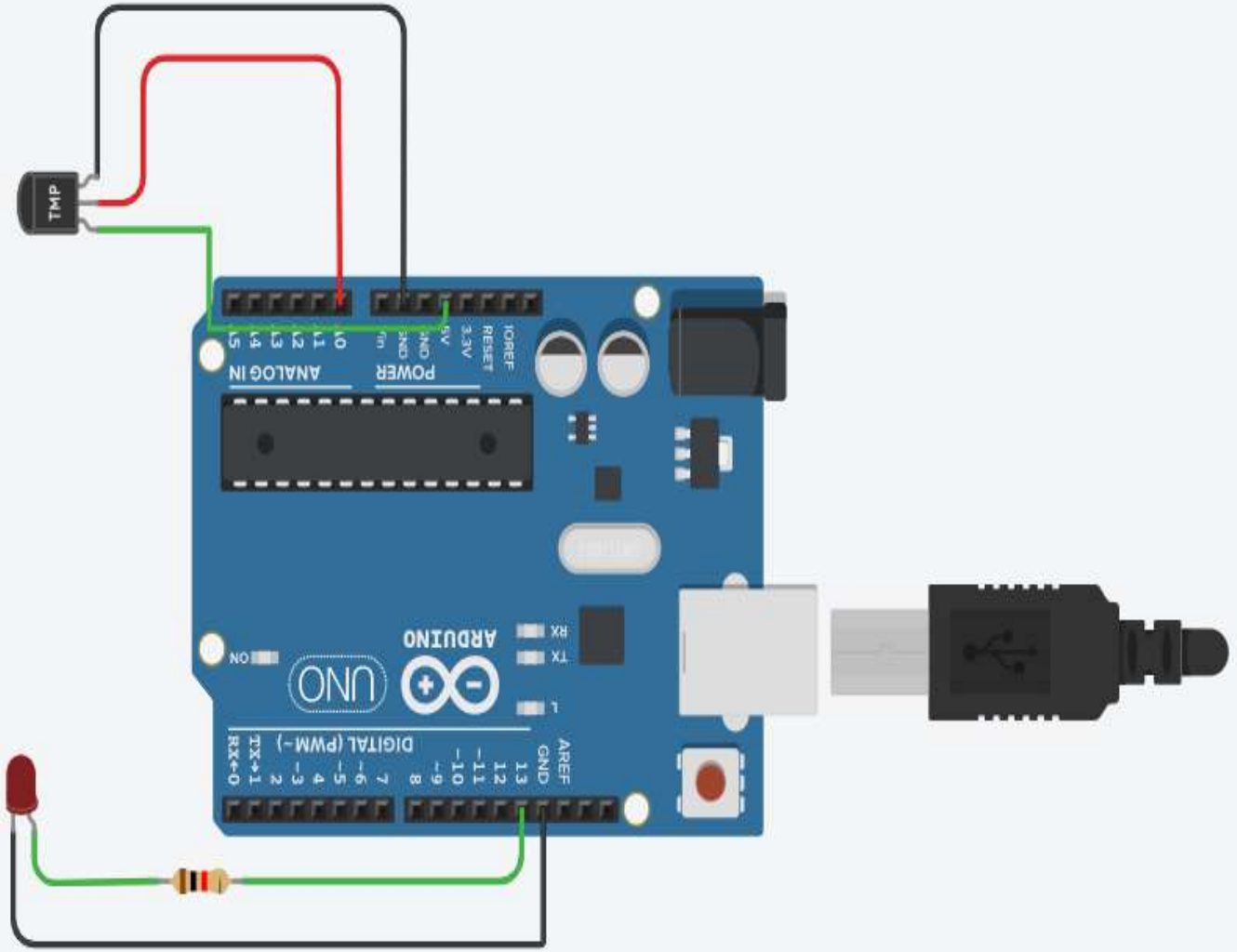
EXAMPLE 2

Code an Arduino board to read a temperature sensor connected to an analog input A0 and display the temperature value in serial window. Each time the value goes beyond 40 degree Celsius an LED connected to a digital output glows as warning.

ANALOG TO DIGITAL (ADC) CONVERTER

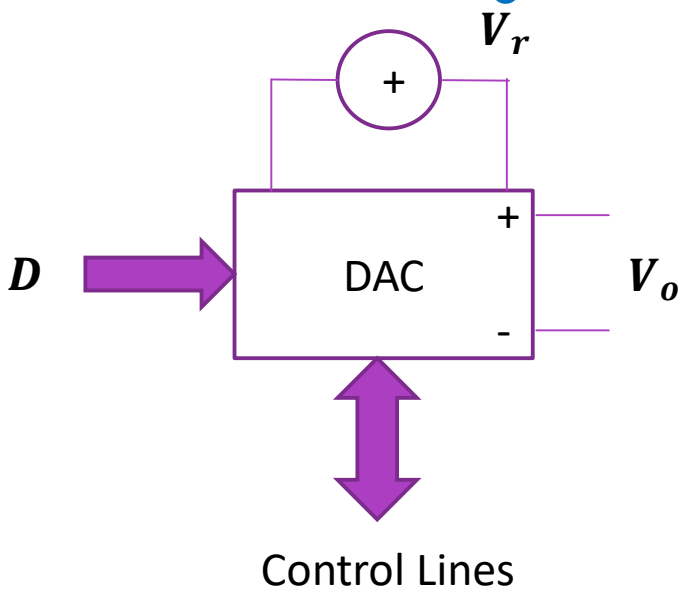
```
const int lm35_pin = A0;  /* LM35 O/P pin */
const int ledPin = 13;
void setup() {
  Serial.begin(9600);
  pinMode(ledPin,OUTPUT);
}

void loop() {
  int temp_adc_val;
  float temp_val;
  temp_adc_val = analogRead(lm35_pin);              /* Read Temperature */
  temp_val = (temp_adc_val * 4.88);    /* Convert adc value to equivalent voltage */
  temp_val = (temp_val/10);              /* LM35 gives output of 10mv/°C */
  if (temp_val > 40) {
      digitalWrite(ledPin, HIGH); // turn LED on:
  } else {
      digitalWrite(ledPin, LOW); // turn LED off:
  }
  Serial.print("Temperature = ");
  Serial.print(temp_val);
  Serial.print(" Degree Celsius\n");
  delay(1000);
}
```

ANALOG TO DIGITAL (ADC) CONVERTER



DIGITAL TO ANALOG (DAC) CONVERTER

- A digital-to-analog converter is a circuit which converts a binary input number into an analog output.
- DAC has a digital input, represented by D, and an analog output, represented by V_o.

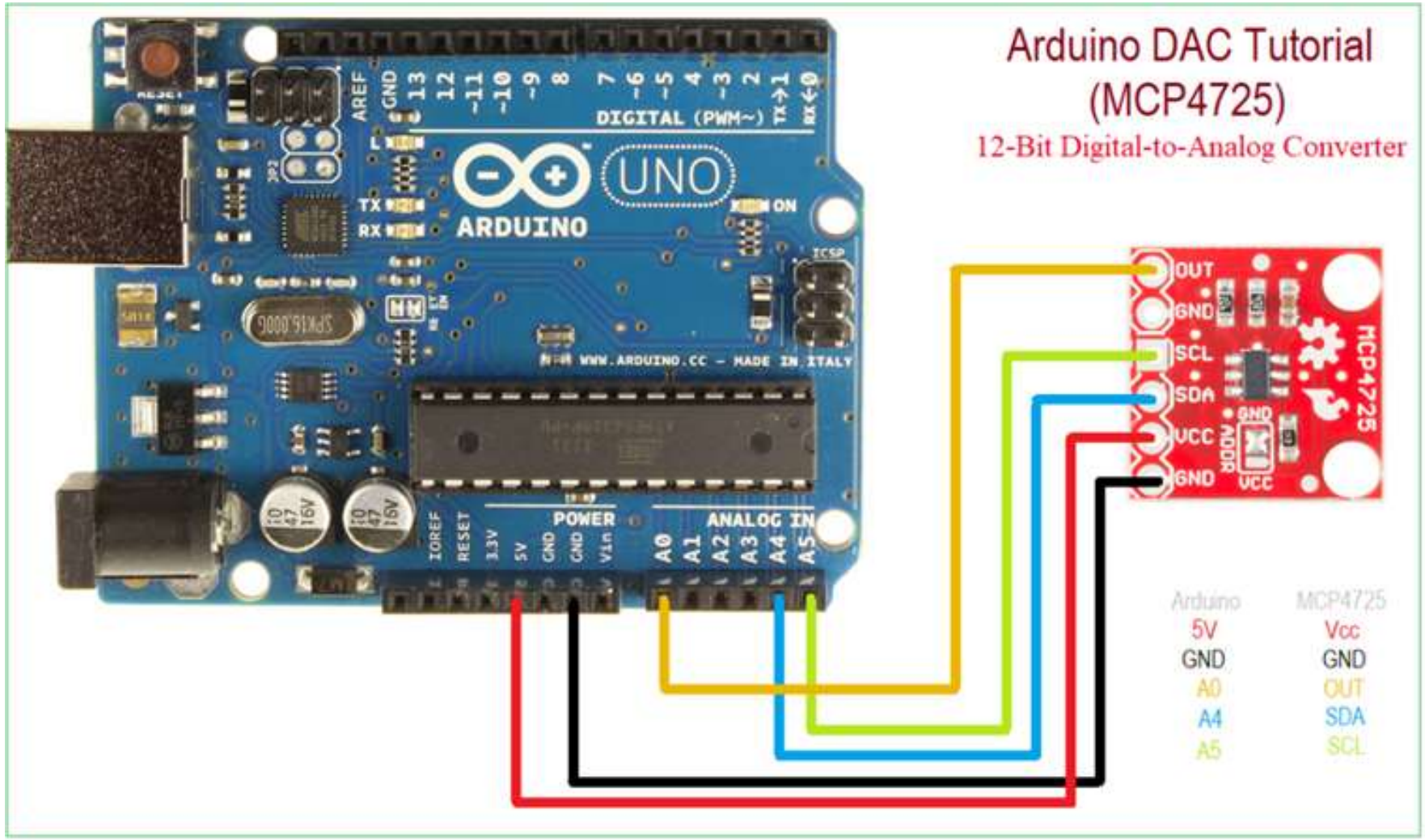


$$V_o = \frac{D}{2^n} \times V_r$$

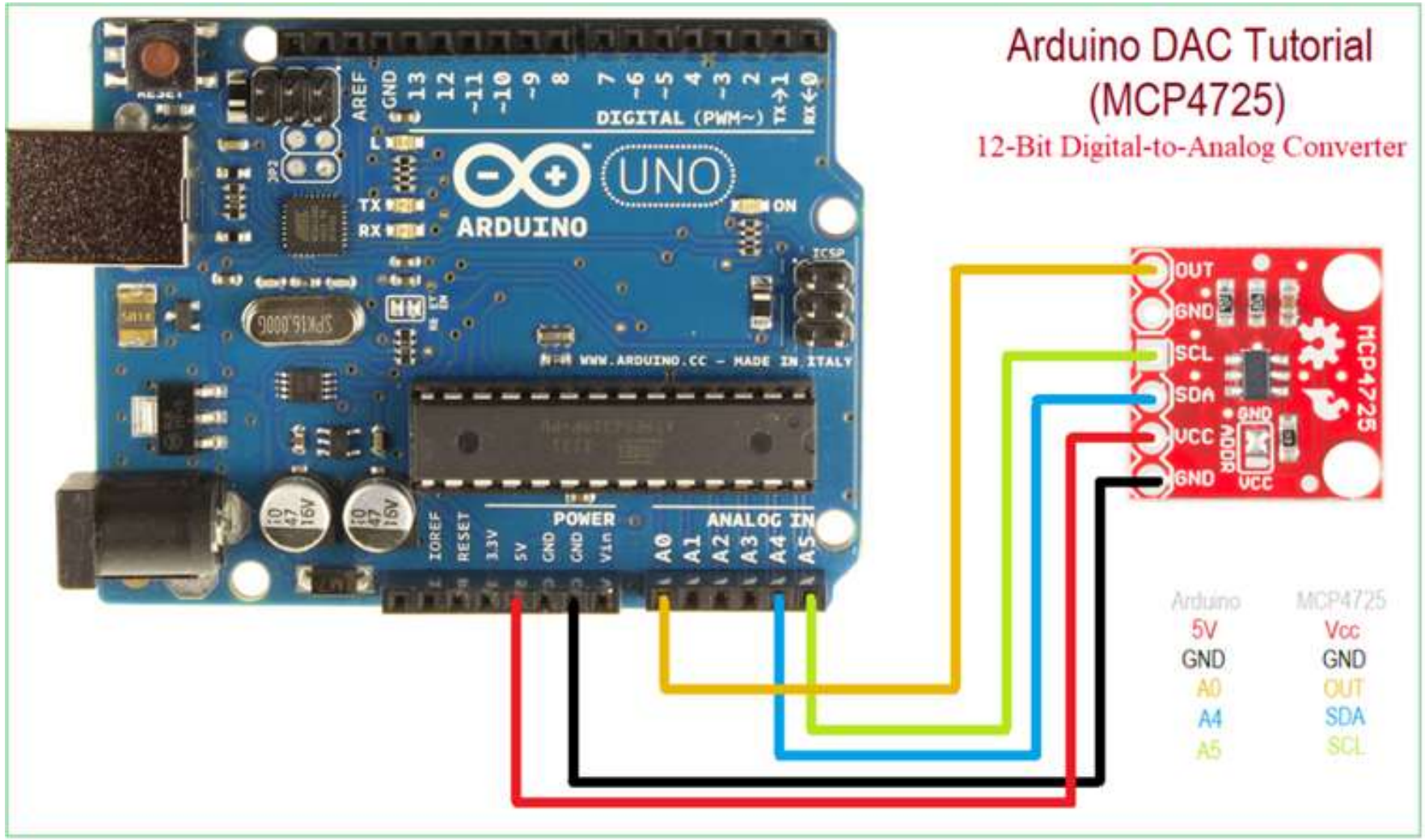
DIGITAL TO ANALOG (DAC) CONVERTER

- For each input digital value, there is a corresponding analog output.
- The number of possible output values is given by 2^n in the range (0 – 4.99V)
- The step size by $V_r/2^n$ this is called the **resolution.**
- The maximum possible output value occurs when $D = (2^n-1)$, so the value of V_r as an output is never quite reached.
  i.e., when D = 1023, V_o = 4.99V

DIGITAL TO ANALOG (DAC) CONVERTER



DIGITAL TO ANALOG (DAC) CONVERTER



For 12-bit DAC,

if A4 (I2C SDA) = 1023,

A0 = (1023/4095) * 5
= 1.25V

PULSE WIDTH MODULATION (PWM)

EXAMPLE 1

Code an Arduino board to control the speed of a DC motor using potentiometer.

PULSE WIDTH MODULATION (PWM)

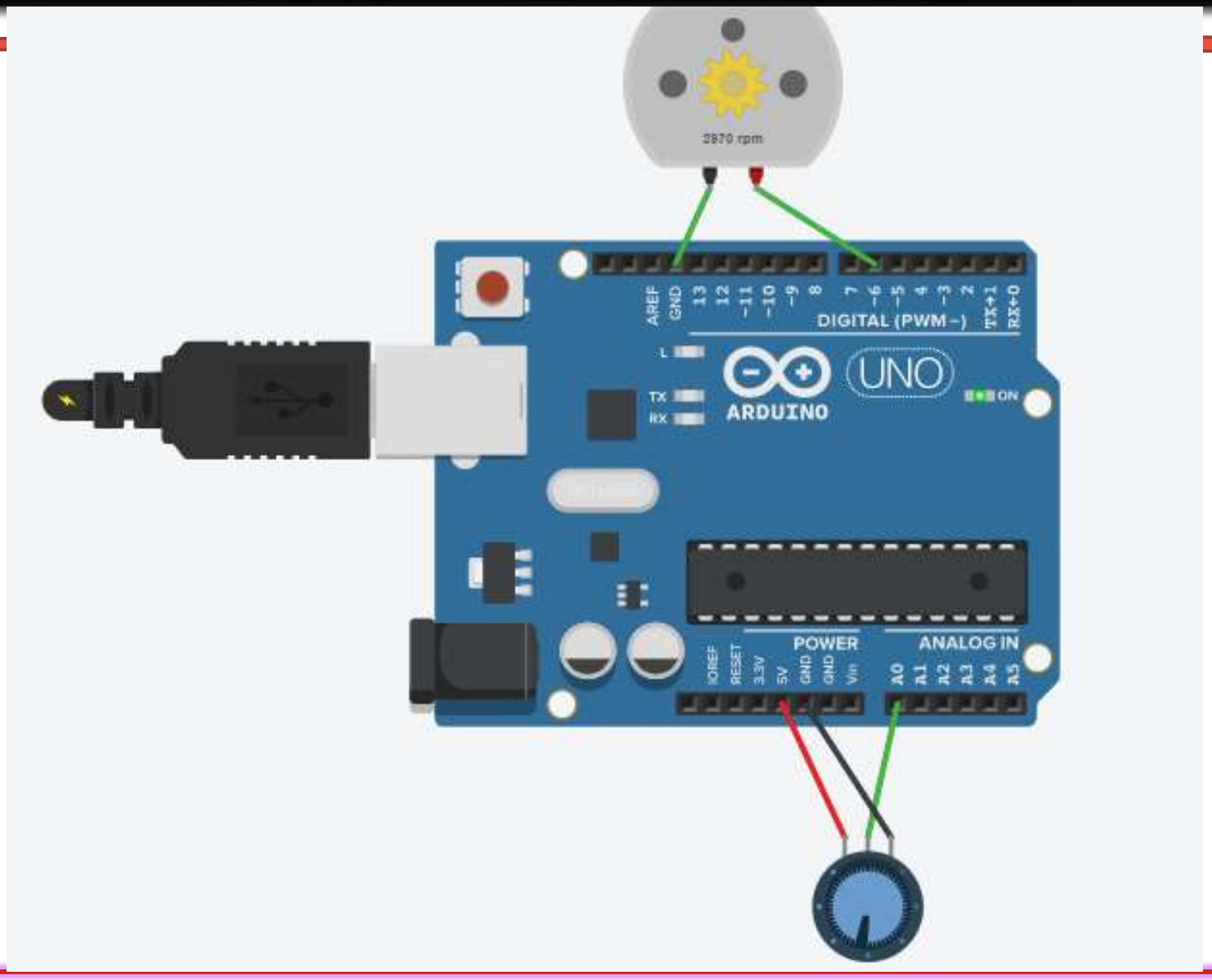
EXAMPLE 1

Code an Arduino board to control the speed of a DC motor using potentiometer.

CODE

```
int potvalue;
void setup()
{
  pinMode(A0, INPUT);
  pinMode(6, OUTPUT);
}

void loop()
{
  potvalue=analogRead(A0)/4;
  analogWrite (6, potvalue);
}
```

ANALOG TO DIGITAL (ADC) CONVERTER



PROGRAMMING TIMERS/COUNTER

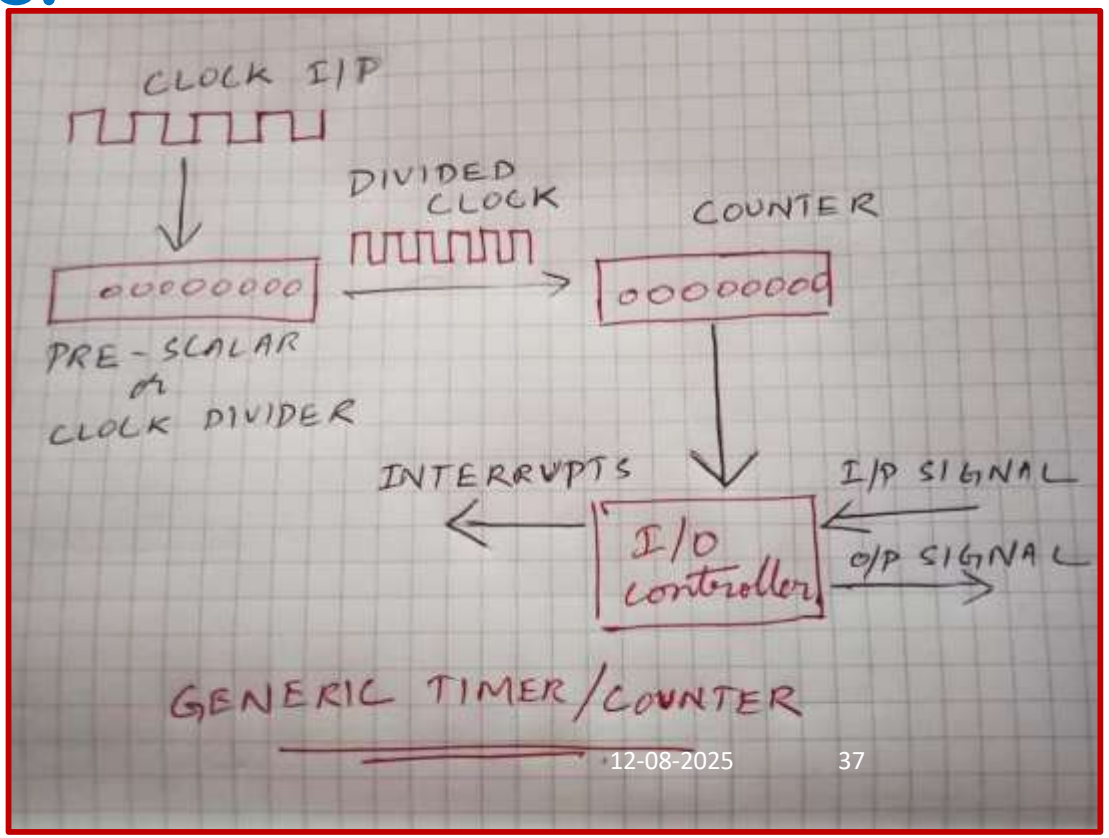
TIMER / COUNTER

- Digital timer/counters are used throughout embedded designs
  - To provide a series of time or count related events within the system
  - With the minimum of processor and software overhead

- Most embedded systems have a time component within them such as
  - Timing references for control sequences
  - To provide system ticks for operating systems and
  - Even the generation of waveforms
  - Serial port baud rate generation and audible tones.

TIMER / COUNTER

- Timers and counters are distinguished from one another largely by their use, not their logic.
  - Timer – Periodic Signal
  - Counter – Aperiodic signal



TIMER / COUNTER

- The central timing is derived from a clock input.
  - Internal to the timer/counter

    (or)
  - External and thus connected via a separate pin.

- The clock may be divided using a simple divider
  - A pre-scalar - effectively divides the clock by the value that is written into the pre-scalar register

- The divided clock is then passed to a counter
  - count-down operation or count-up operation

TIMER / COUNTER

- When a zero count is reached, an event occurs.
  - such as an interrupt of an external line changing state.

- The final block is an I/O control block
  - It generates interrupts
  - Can control the counter based on external signals

TIMER / COUNTER

Timers on the Arduino

- The Arduino Uno has 3 timers: Timer0, Timer1 and Timer2
  1. Timer 0 – 8 bit timer ; Functions – delay(), millis().
  2. Timer 1 – 16 bit timer; Servo library
  3. Timer 2 – 8 bit timer; Function – tone()

- Arduino Uno has a 16Mhz internal clock.
- It takes 62 nano seconds for a single count.
- Update the pre-scalar to alter the frequency and various counting modes.
- Generate interrupts when the timer reaches a specific count

TIMER / COUNTER

Control Registers

- Timer/Counter Control Registers A: (TCCRnA)
- Timer/Counter Control Registers B: (TCCRnB)
- Timer CounNT: TCNTn
- Output Compare Register A: OCRnA
- Output Compare Register B: OCRnB

n= C/T number 0,1,2

http://www.avrbeginners.net/architecture/timers/timers.html

TIMER / COUNTER

Control Registers

- TCCRnA: sets mode and output compare behavior
- TCCRnB: sets prescaler and waveform generation
- TCNTn: holds the current timer/counter value
- OCRnA: triggers event when TCNTn = OCRnA
- OCRnB: triggers event when TCNTn = OCRnB

n= C/T number 0,1,2

http://www.avrbeginners.net/architecture/timers/timers.html

TIMER / COUNTER

Timer/Counter Modes

- Normal : Counter count up to maximum value and reset to 0
- Phase correct PWM: symmetric with respect to system clock using PWM
- Clear Time on Compare (CTC): Used in interrupt generation
- Fast PWM: PWM goes as fast as clock but not synchronized with timing clock

http://www.avrbeginners.net/architecture/timers/timers.html

TIMER / COUNTER

Timers on the Arduino

- Timer/Counter Control Registers (TCCRnA/B)
  - Controls the mode of timer
  - Contains the pre-scalar values of timers
  - Prescalars values 1, 8, 64, 256, 1024

- Timer/Counter Register (TCNTn)
  - Control the counter value
  - Set the pre-loader value

To calculate preloaded value for timer1 for time of 2 Sec:

TCNT1= 65535-(16x10^6x time(s)/ Prescalar value)

TCNT1 = 65535 – (16x 10^6x2 / 1024)

= 34285

TIMER / COUNTER

Timers on the Arduino

- delay(ms)
- Pauses the program for the amount of time.
- Time is specified in milliseconds
- A delay of 1000 ms equals 1 s

delay(ms)

Parameters
ms: the number of milliseconds to pause (unsigned long)

Returns
None

TIMER / COUNTER

Timers on the Arduino

- delayMicroseconds(us)
- Used to delay for a much shorter time.

- A time period of 1000 µs would equal 1 ms.

delayMicroseconds(us)

Parameters
us: the number of microseconds to pause (unsigned int)

Returns
None

TIMER / COUNTER

Timers on the Arduino

- millis()
- Use of one of internal timer1 to maintain a running counter.
- How many milliseconds the microcontroller has been running since the last time it was turned on or reset.
- It returns a value in milliseconds

millis()

Parameters
None

Returns
Number of milliseconds since the program started (unsigned long)

TIMER / COUNTER

Timers on the Arduino

- micros()
- Returns the value in microseconds.

micros()

Parameters
None

Returns
Number of microseconds since the program started (unsigned long)

TIMER / COUNTER

EXAMPLE

Code an Arduino board to read a pushbutton connected to a digital input to display millis() when ON and display micros() when OFF. Blink an LED connected to a digital output with 500 ms delay.

TIMER / COUNTER

EXAMPLE

Code an Arduino board to read a pushbutton connected to a digital input to display millis() when ON and display micros() when OFF. Blink an LED connected to a digital output with 500 ms delay.

Note:

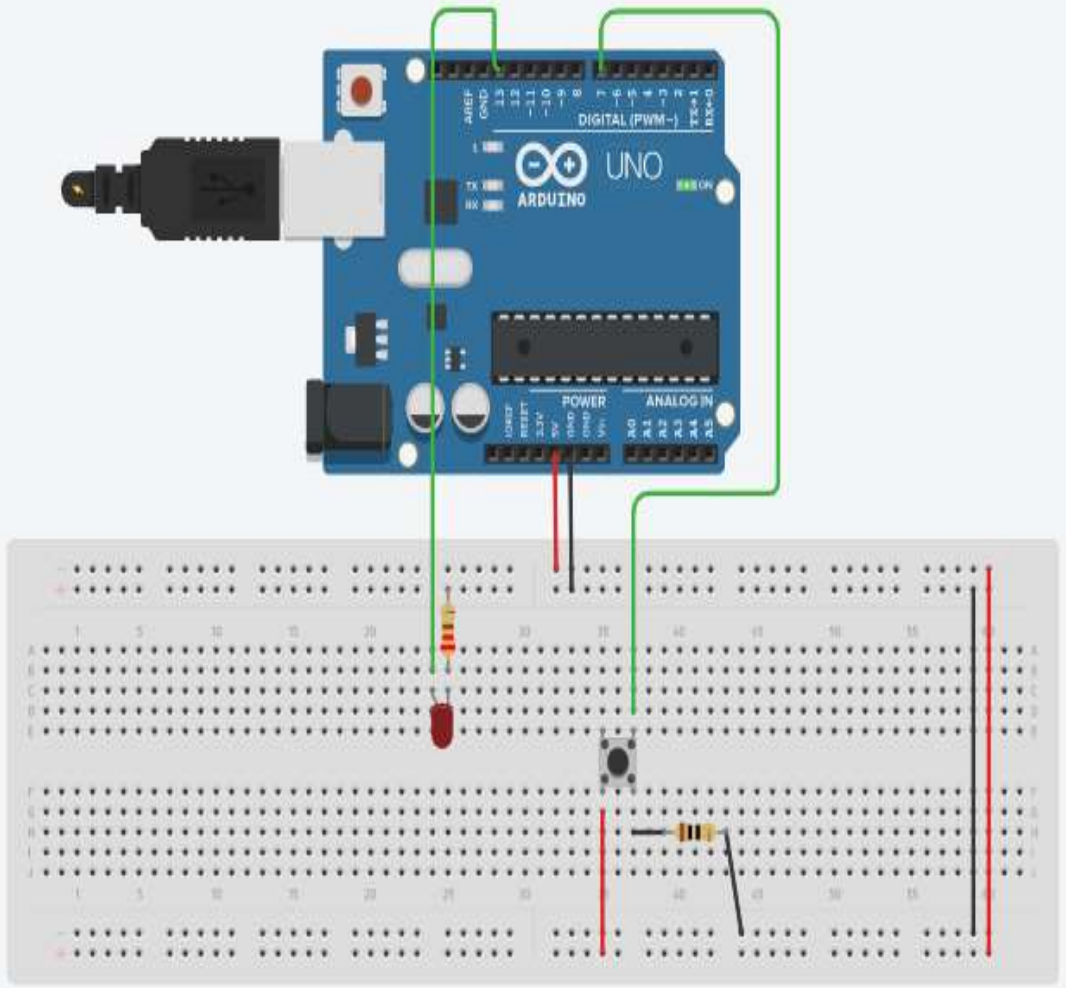
For pushbutton, initially buttonState will be HIGH. This is because, internal pull-up resistor will be UP.

Once it is pressed down, INPUT_PULLUP is down, causing buttonState to be LOW. This gets reverted to HIGH, once button press is released.

TIMER / COUNTER

```
#define LED 13
int buttonState = 0;
void setup() {
        Serial.begin(9600);
        pinMode(LED, OUTPUT);
        pinMode(7, INPUT);}
void loop() {
        digitalWrite(LED, HIGH);
        delay(500);
        digitalWrite(LED, LOW);
        delay(500);
        buttonState = digitalRead(7);
        if(buttonState==LOW) {
                        Serial.write("micros()=");
                        Serial.println(micros());
                        }else {
                        Serial.write("millis()=");
                        Serial.println(millis());
        }}
```

TIMER / COUNTER



Serial Monitor

```
millis()=1000
millis()=2000
millis()=3002
millis()=4003
millis()=5005
millis()=6005
millis()=7007
micros()=8009064
micros()=9010636
micros()=10012212
micros()=11013904
micros()=12015600
millis()=13017
millis()=14018
millis()=15019
```

TIN KER CAD

PWM GENERATION

PULSE WIDTH MODULATION (PWM)

- Pulse width modulation (PWM) create simple square wave varying the duty cycle.
- It used to control an analog variable, usually voltage or current.
- Used in a variety of applications, ranging from telecommunications to robotic control.
- Arduino has six PWM outputs (Pin11, Pin10, Pin9, Pin6, Pin5, Pin3).

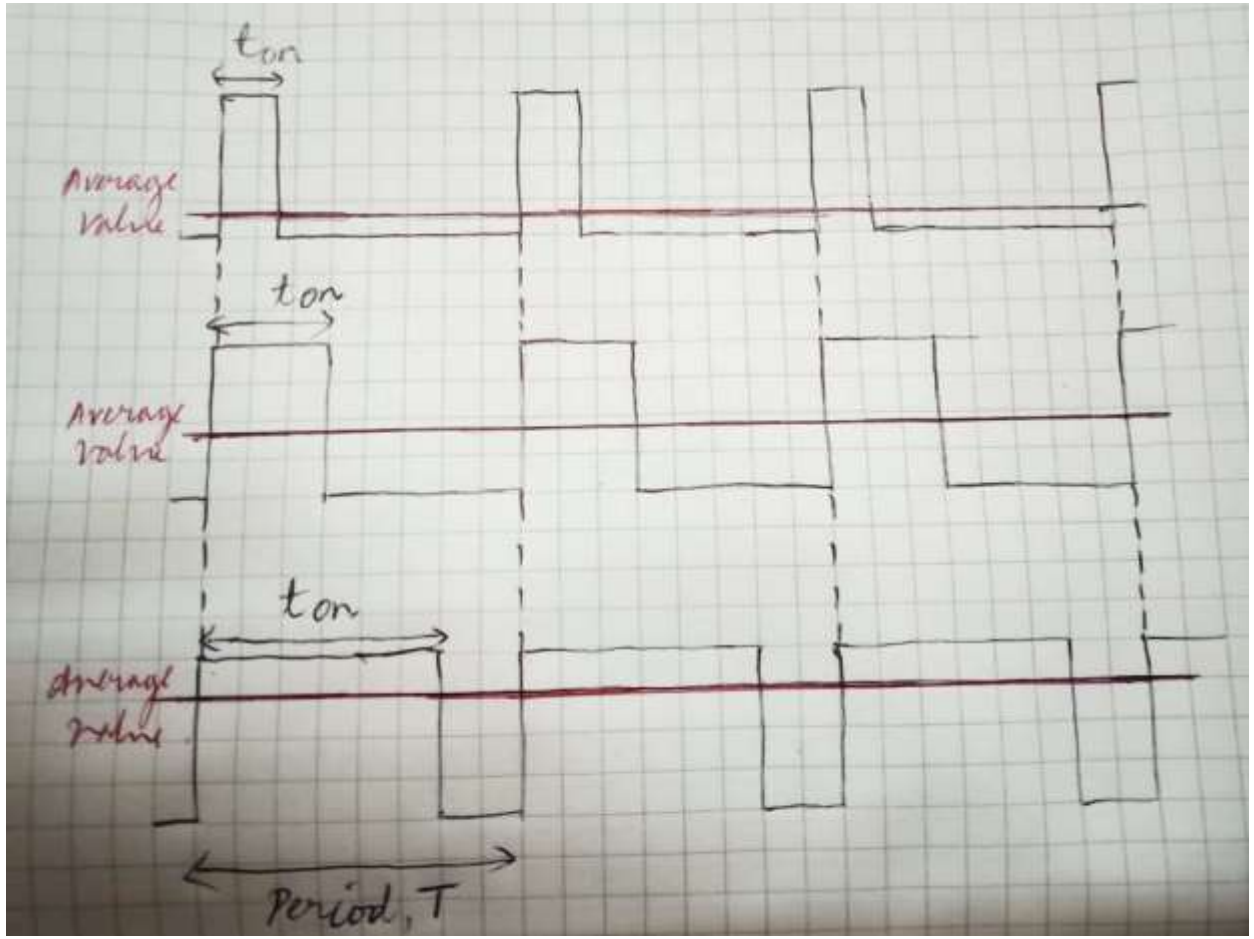
PULSE WIDTH MODULATION (PWM)

- PWM depends on pulse width and duty cycle.
- The pulse width (also called a period) is a short duration of time in which the duty cycle will operate.
- The **duty cycle** is the proportion of time that the pulse is "on" or "high" and is expressed as a percentage.

$$Duty\ cycle = \frac{pulse\ on\ time}{pulse\ period} \times 100\% = \frac{t_{on}}{T} \times 100\%$$

- 100% duty cycle - **continuously on**
- 0% duty cycle - **continuously off**

PULSE WIDTH MODULATION (PWM)



PULSE WIDTH MODULATION (PWM)

- DC motor control is a very common task in robotics.
- Speed of DC motor is proportional to the applied DC voltage.
- Conventional DAC output is used,
  - drive it through an expensive and bulky power amplifier
  - use the amplifier output to drive the motor
- Alternatively, a PWM signal can be used
  - to drive a power transistor directly

PULSE WIDTH MODULATION (PWM)

Analog I/O

analogWrite(pin,value)

- Write an analog output on a digital pin.
- This is called Pulse Width Modulation.
- PWM uses digital signals to control analog devices.
- Digital Pins # 3, # 5, #6, # 9, # 10, and # 11 can be used as PWM pins.
- No need to configure pin as an OUTPUT.

analogWrite(pin,value)

Parameters pin: the number of the pin you want to write
value: the duty cycle between 0 (always off, 0%) and 255 (always on, 100%)

Returns
None

PULSE WIDTH MODULATION (PWM)

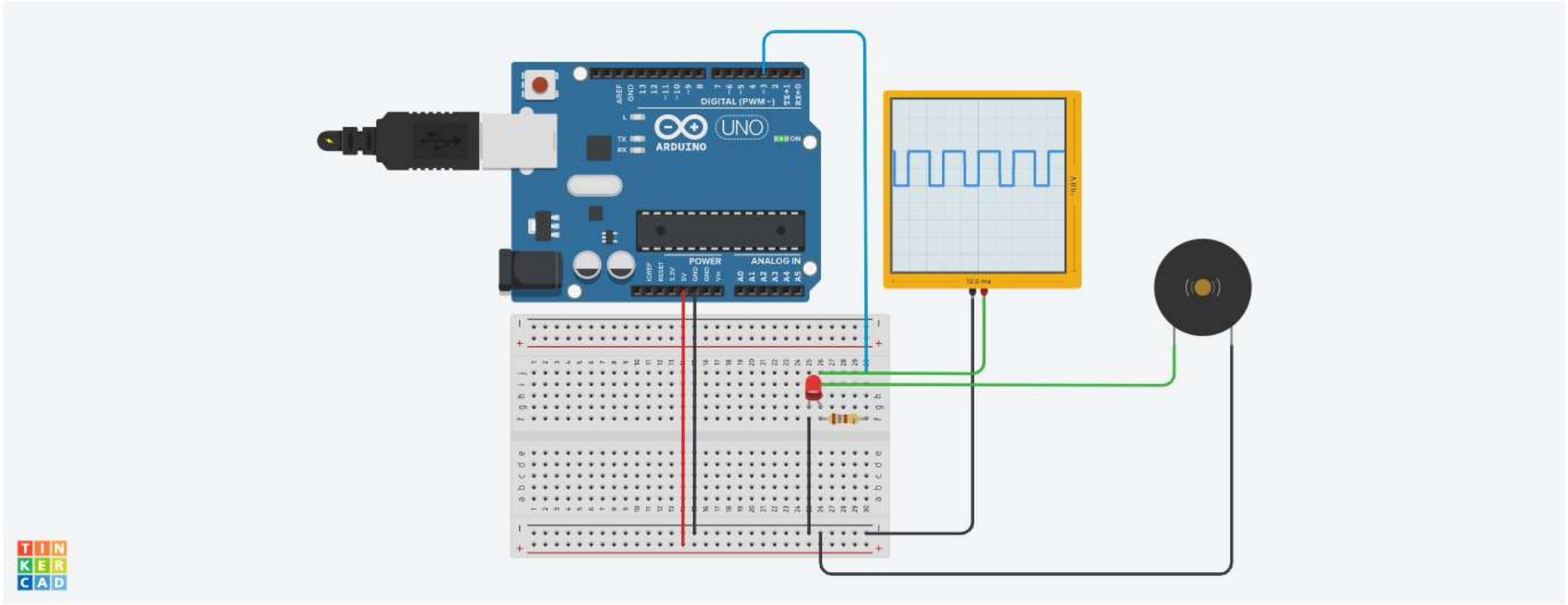
EXAMPLE 1

Code an Arduino board to control the brightness of an LED. Monitor the PWM waveform in an oscilloscope and interface a buzzer to listen to different tones.

PULSE WIDTH MODULATION (PWM)

```
int britns = 0;
void setup()
{   pinMode(3, OUTPUT); }
void loop()
{
  for (britns = 0; britns <= 255; britns += 5) {
    analogWrite(3, britns);
    delay(30); // Wait for 30 millisecond(s)
  }
  for (britns = 255; britns >= 0; britns -= 5) {
    analogWrite(3, britns);
    delay(30); // Wait for 30 millisecond(s)
  }
}
```

ANALOG TO DIGITAL (ADC) CONVERTER



PULSE WIDTH MODULATION (PWM)

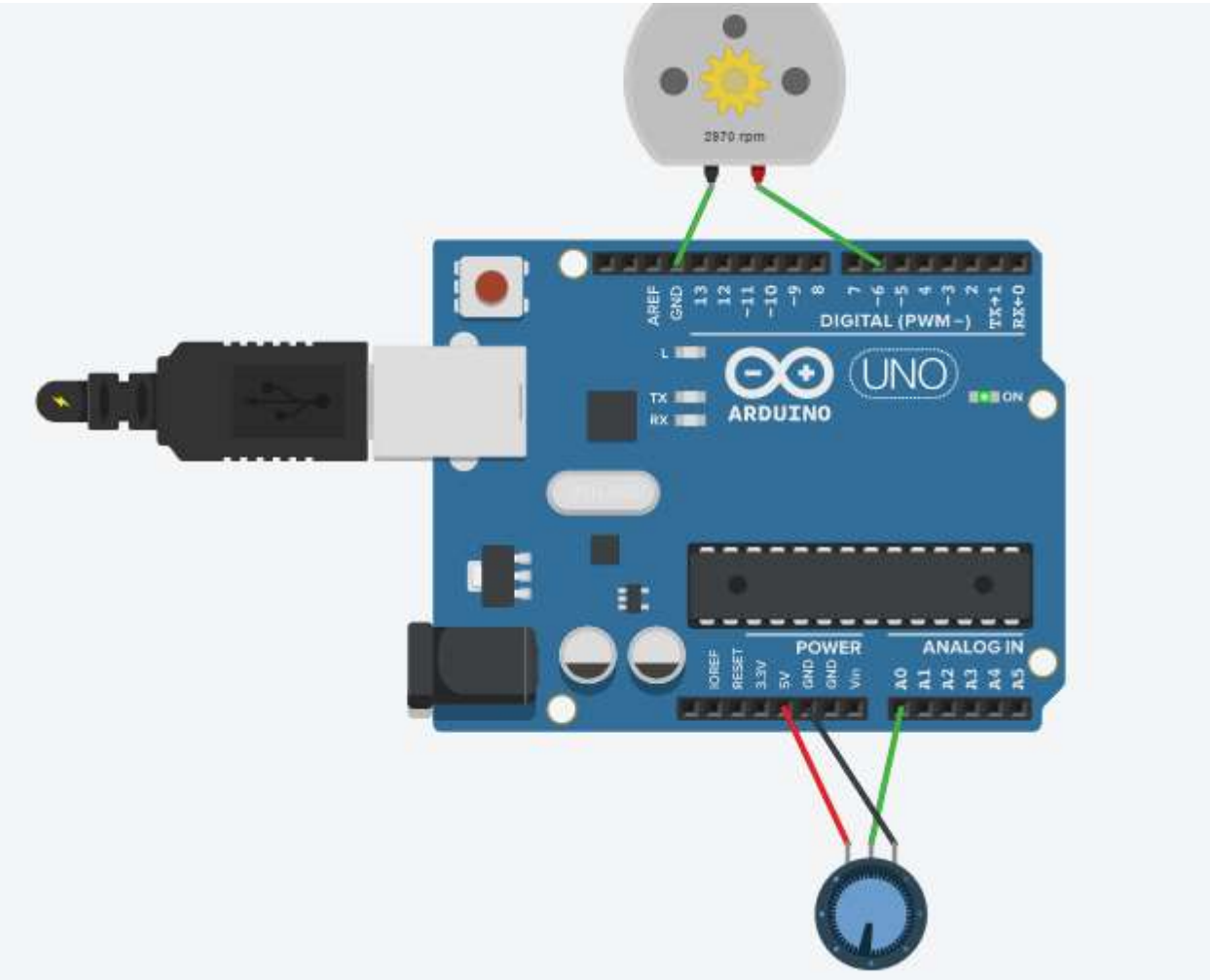
EXAMPLE 1

Code an Arduino board to control the speed of a DC motor using potentiometer.

CODE

```
int potvalue;
void setup()
{
  pinMode(A0, INPUT);
  pinMode(6, OUTPUT);
}

void loop()
{
  potvalue=analogRead(A0)/4;
  analogWrite (6, potvalue);
}
```

ANALOG TO DIGITAL (ADC) CONVERTER

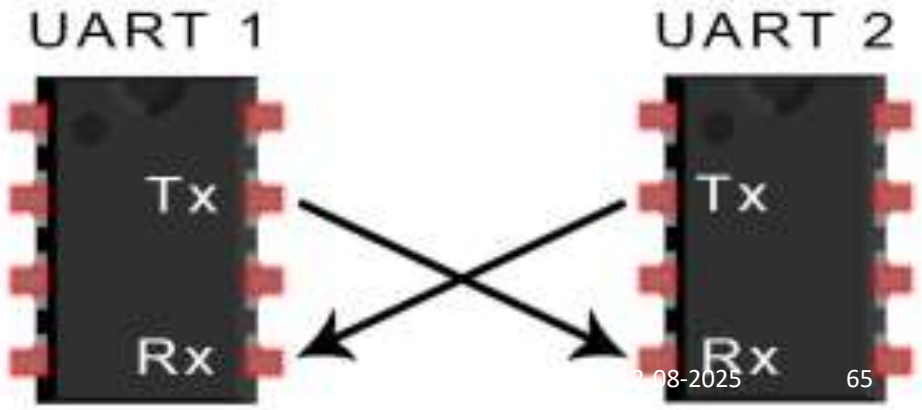


UART

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- UART stands for Universal Asynchronous Receiver/Transmitter
- It is used to transmit and receive serially
- Two wires are required to transmit and receive data Tx and Rx Pin



https://www.circuitbasics.com/basics-uart-communication/

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- UARTs has no clock signal and follows asynchronous data communication
- Start and stop bits are added to data packet for identifying beginning and end of data packet
- Start bit and stop bit are used instead of clock signal

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- After detecting start bit UART starts to read the incoming data packet
- A specific frequency is used for data communication known as baud rate
- Baud rate is expressed as bits per second (bps). It is a measure of speed of data transfer.
- Both receiving and transmitting UART should work in same baud rate

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

Parameters	Specification
Wires used	2
Maximum speed	115200 baud rate
Synchronous ?	No
Serial?	Yes
Max number of master	1
Max number of slave	1

https://www.circuitbasics.com/basics-uart-communication/

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS

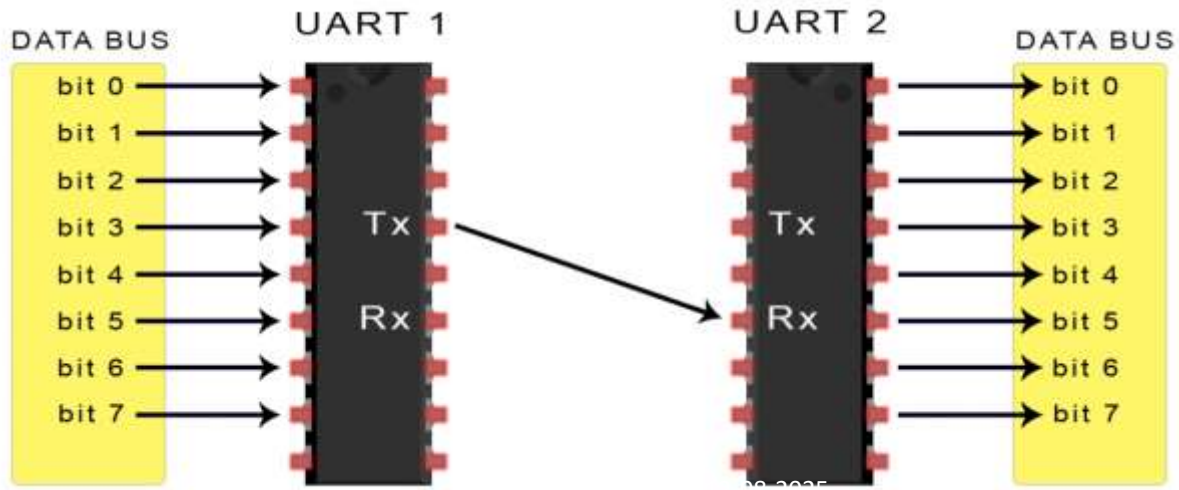
- In UART, data is transmitted in serial from data bus to transmitting UART
- The transmitting UART adds a start bit, stop bit and a parity bit to parallel data from data bus
- the data packet is transmitted serially from Tx to RX pin
- The Rx pin reads the data and converts in parallell by removing other bits

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS

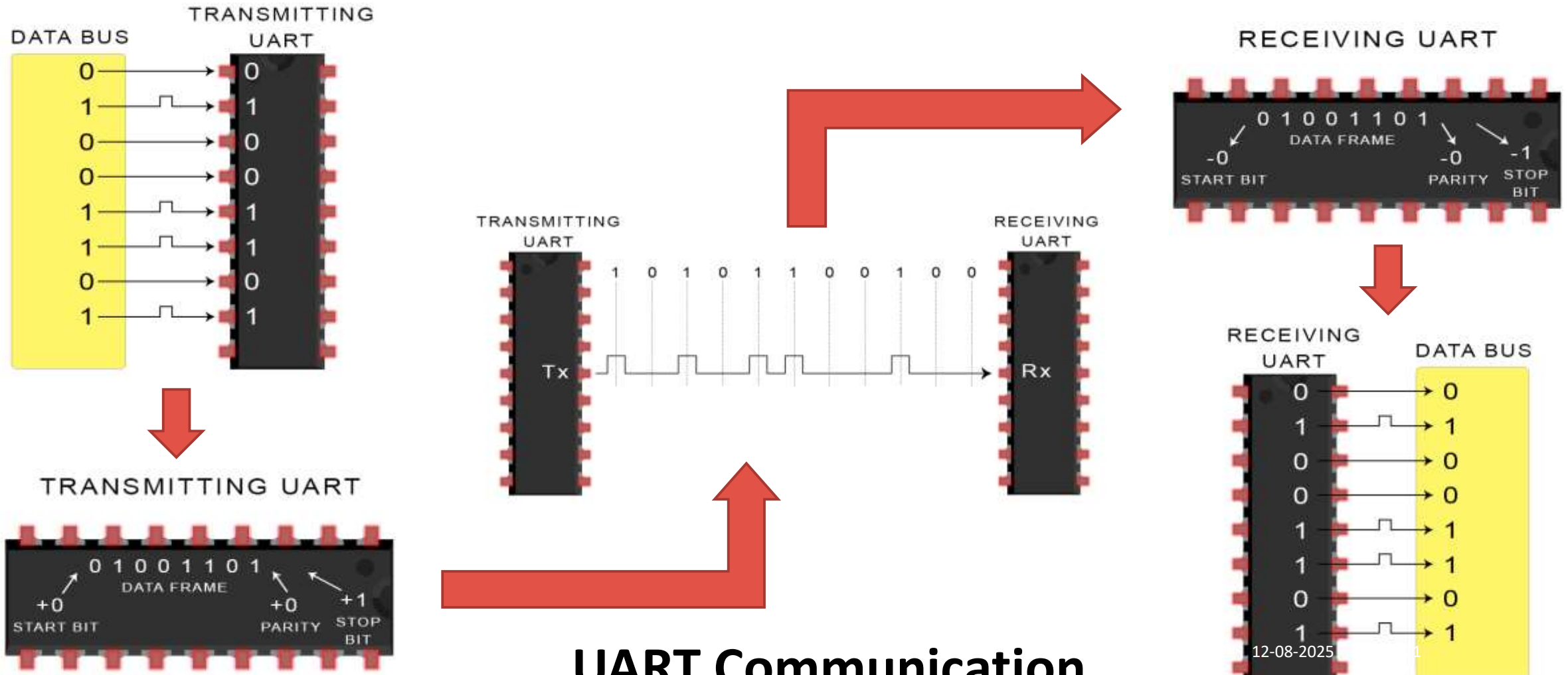
Frame Format

1 start bit	5 to 9 data bit	0 or 1 parity bit	1 to 2 stop bits



https://www.circuitbasics.com/basics-uart-communication/

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

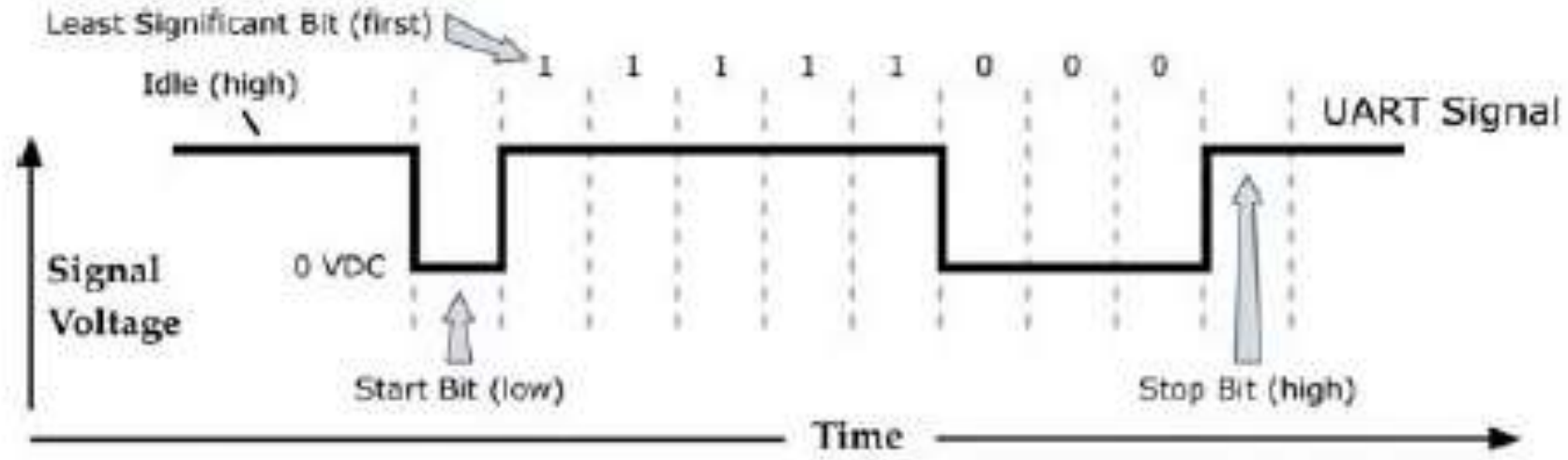


UART Communication

https://www.circuitbasics.com/basics-uart-communication/

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS



A common asynchronous serial data format

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

UART IN ARDUINO

- UART allows the Atmega chip to do serial communication while working on other tasks, through 64 byte serial buffer.
- All Arduino boards have at least one serial port (also known as a UART or USART): **Serial**.
- Serial is used for communication between the Arduino board and a computer or other devices.
- It communicates on digital pins 0 (RX) and 1 (TX) as well as with the computer via USB. Thus, if you use these functions, you cannot also use pins 0 and 1 for digital input or output.

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino functions for USB serial communication

- if (Serial) – indicates whether or not the USB serial connection is open
- Serial.available()-Get the number of bytes available for reading from the serial port.
- Serial.begin()-Sets the data rate in bits per second (baud) for serial data transmission.
- Serial.end()-Disables serial communication, allowing the RX,TX to be used for input & output.
- Serial.find()-reads data from the serial buffer until the target is found.
- Serial.println()-Prints data as ASCII text followed by a carriage return and newline character.
- Serial.read()-Reads incoming serial data.
- Serial.readString() - reads characters from the serial buffer into a String.
- Serial.write() - Writes binary data to serial port. This data is sent as a byte or series of bytes

For other functions refer the URL:
https://www.arduino.cc/en/pmwiki.php?n=Reference/serial#:~:text=Serial%20is%20used%20for%20communication,with%20the%20computer%20via%20USB.

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- **EXAMPLE-1: Print data received through serial communication on to the serial monitor of Arduino**

```
void setup() {
  Serial.begin(9600); //set up serial library baud rate to 9600
}

void loop() {
  if(Serial.available()) //if number of bytes (characters) available for reading from serial port
{
Serial.print("I received:"); //print I received
Serial.write(Serial.read()); //send what you read
  }
}
```

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- **EXAMPLE-1: Arduino code for serial interface to blink switch ON LED when "a" is received on serial port**

```
int inByte; // Stores incoming command
void setup() {
Serial.begin(9600);
pinMode(13, OUTPUT); // Led pin
Serial.println("Ready"); // Ready to receive commands
}
void loop() {
if(Serial.available() > 0) { // A byte is ready to receive
inByte = Serial.read();
if(inByte == 'a') { // byte is 'a'
digitalWrite(13, HIGH);
Serial.println("LED - On");
}
else { // byte isn't 'a'
digitalWrite(13, LOW);
Serial.println("LED - off");
} } }
```

This function can be used
if(Serial.find("a"))

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino SoftwareSerial Library

- The SoftwareSerial library has been developed to allow serial communication on other digital pins of the Arduino
- Uses software to replicate the functionality of the hardwired RX and TX lines hence the name "SoftwareSerial".
- It is possible to have multiple software serial ports with speeds up to 115200 bps.
- This can be extremely helpful when the need arises to communicate with two or more serial enabled devices
- Limitations:
  - maximum RX speed is 57600bps and RX doesn't work on Pin 13
  - If using multiple software serial ports, only one receive data at a time.

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino functions for UART communication (SoftwareSerial Library)

SoftwareSerial() – Need to enable serial communication

available() – gets the no of bits available for reading from serial port

begin() – sets the speed of serial communication

overflow() – checks the serial buffer overflow has occurred

peek() – returns the character received in the serial port

read() - returns the character that was received on the Rx pin of serial port

Print() – works same as serial.print

Listen() – enables the selected serial port to listen

Write() – print data to transmit pin of software serial

Refer. URL: https://www.arduino.cc/en/Reference/SoftwareSerial

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- **EXAMPLE-3:** **Arduino code to Receives from the hardware serial, sends to software serial and Receives from software serial, sends to hardware serial.**

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX
void setup() {
  Serial.begin(57600); // Open serial communications and wait for port to open:
 while (!Serial) {
  ; // wait for serial port to connect. Needed for native USB port only
 }
 mySerial.begin(4800); // set the data rate for the SoftwareSerial port
 mySerial.println("Hello, world?");
}
void loop() { // run over and over
 if (mySerial.available()) {
  Serial.write(mySerial.read());
 }
 if (Serial.available()) {
  mySerial.write(Serial.read());
 } }
```

Sensor & Actuators

Sensors

- Electronic elements
- Converts physical quantity/ measurements into electrical signals
- Can be analog or digital

Sensor & Actuators

Types of Sensors

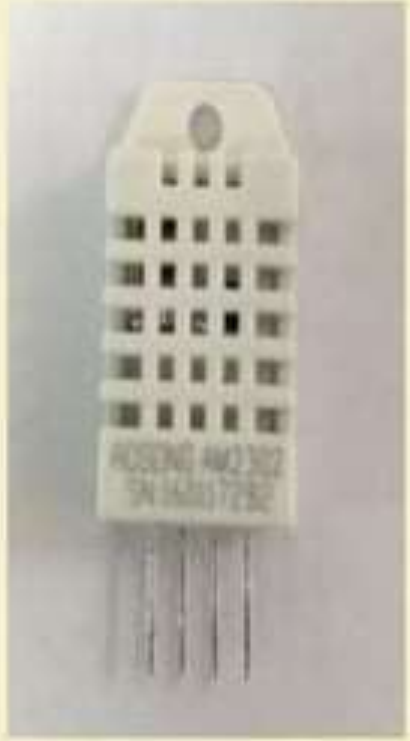
Some commonly used sensors:

- Temperature
- Humidity
- Compass
- Light
- Sound
- Accelerometer

Sensor & Actuators

Sensor Interface with Arduino

- Digital Humidity and Temperature Sensor (DHT)
- PIN 1, 2, 3, 4 (from left to right)
  - PIN 1- 3.3V-5V Power supply
  - PIN 2- Data
  - PIN 3- Null
  - PIN 4- Ground



Sensor & Actuators

DHT Sensor Library

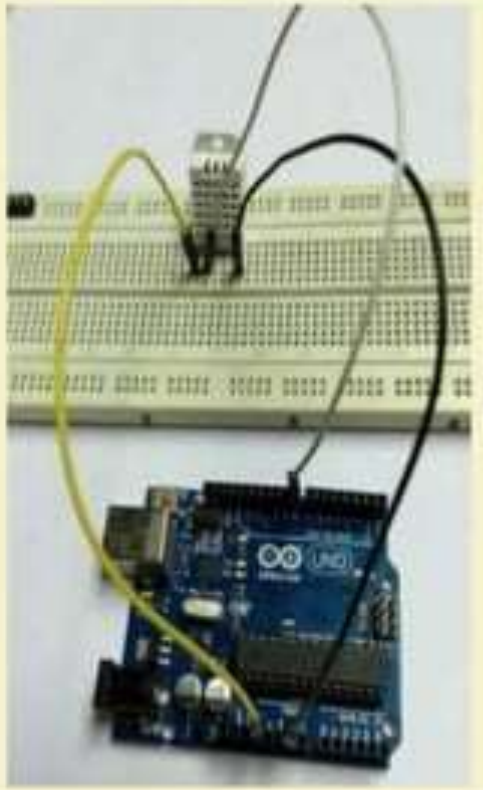
- Arduino supports a special library for the DHT11 and DHT22 sensors
- Provides function to read the temperature and humidity values from the data pin

```
dht.readHumidity()
dht.readTemperature()
```

Sensor & Actuators

Connection

- Connect pin 1 of the DHT to the 3.3 V supply pin in the board
- Data pin (pin 2) can be connected to any digital pin, here 12
- Connect pin 4 to the ground (GND) pin of the board

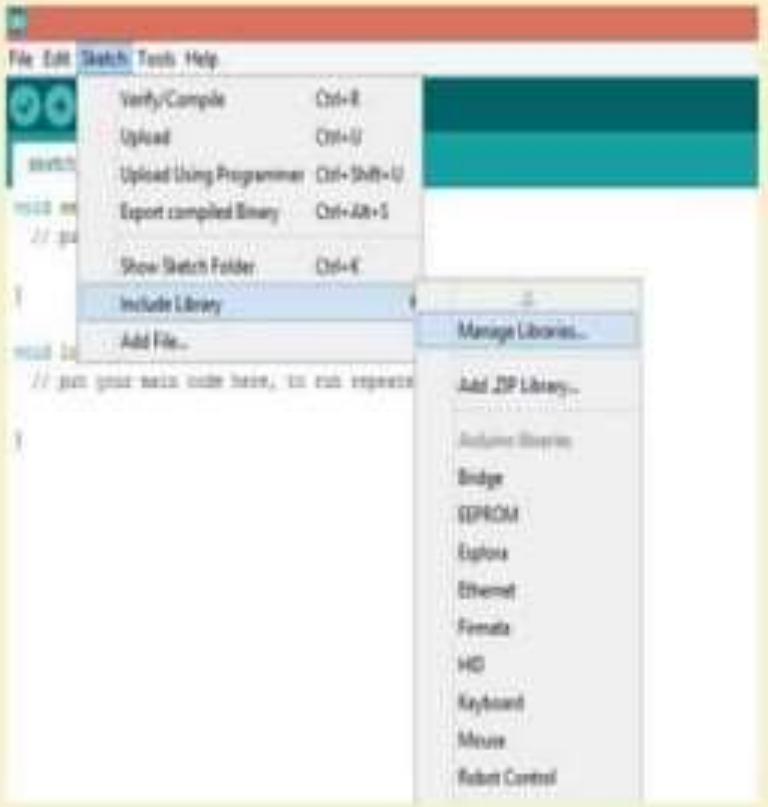


Sensor & Actuators

Sketch: DHT_SENSOR

Install the DHT Sensor Library

- Go to Sketch -> Include Library -> Manage Library



Sensor & Actuators

Sketch: DHT_SENSOR (contd..)

- Search for DHT SENSOR
- Select the "DHT sensor library" and install it

Sensor & Actuators

Sketch: DHT_SENSOR (contd..)

```
#include <DHT.h>;
DHT dht(8, DHT22);     //Initialize DHT sensor
float humidity;        //Stores humidity value
float temperature;     //Stores temperature
value
void setup()
{
 Serial.begin(9600);
 dht.begin();
}
```

```
void loop()
{
  //Read data from the sensor and store it to variables
humidity and temperature
  humidity = dht.readHumidity();
  temperature= dht.readTemperature();
  //Print temperature and humidity values to serial
monitor
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.print("%, Temperature: ");
  Serial.print(temperature);
  Serial.println(" Celsius");
  delay(2000); //Delay of 2 seconds
```

Sensor & Actuators

Sketch: DHT_SENSOR (contd..)

Sensor & Actuators

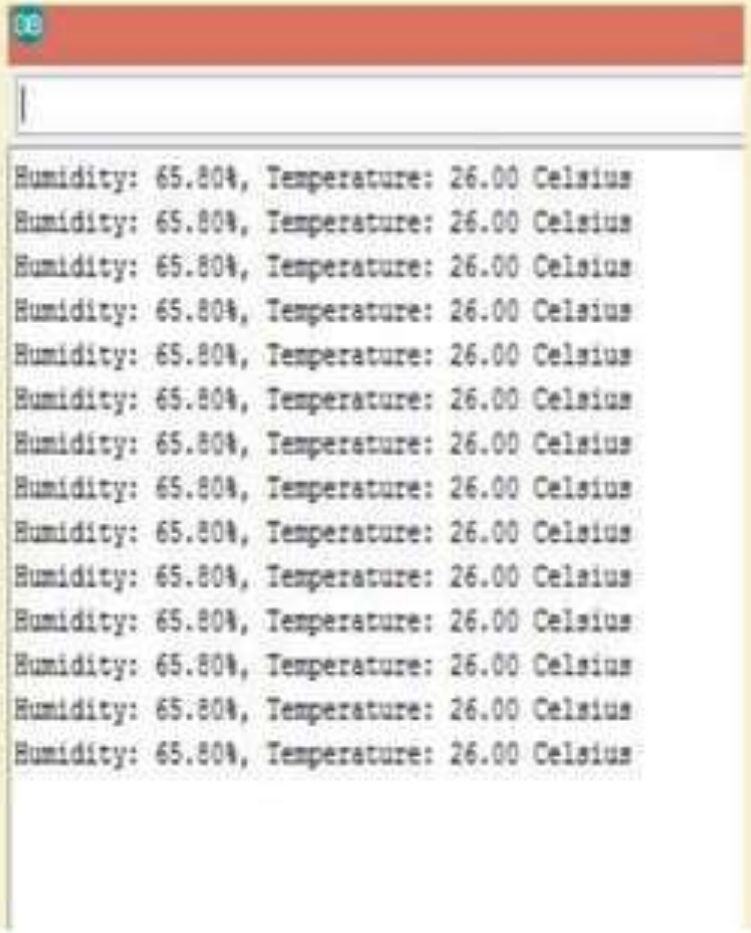
Sketch: DHT_SENSOR (contd..)

- Connect the board to the PC
- Set the port and board type
- Verify and upload the code

Sensor & Actuators

Output

The readings are printed at a delay of 2 seconds as specified by the delay() function



Sensor & Actuators

Actuators

- Mechanical/Electro-mechanical device
- Converts energy into motion
- Mainly used to provide controlled motion to other components

Basic Working Principle

Uses different combination of various mechanical structures like screws, ball bearings, gears to produce motion.

Sensor & Actuators

Types of Motor Actuators

- Servo motor
- Stepper motor
- Hydraulic motor
- Solenoid
- Relay
- AC motor

Sensor & Actuators

Servo Motor

- High precision motor
- Provides rotary motion 0 to 180 degree
- 3 wires in the Servo motor
  - Black or the darkest one is Ground
  - Red is for power supply
  - Yellow for signal pin

Sensor & Actuators

Servo Library on Arduino

- Arduino provides different library- SERVO to operate the servo motor
- Create an instance of servo to use it in the sketch

```
Servo myservo;
```

Sensor & Actuators

Sketch: SERVO_ACTUATOR

```
#include <Servo.h>
//Including the servo library for the program
int servoPin = 12;


Servo ServoDemo; // Creating a servo object
void setup() {
  // The servo pin must be attached to the servo
before it can be used
  ServoDemo.attach(servoPin);
}
```

```
void loop(){
  //Servo moves to 0 degrees
  ServoDemo.write(0);
  delay(1000);

  // Servo moves to 90 degrees
  ServoDemo.write(90);
  delay(1000);

  // Servo moves to 180 degrees
  ServoDemo.write(180);
  delay(1000);
}
```

Sensor & Actuators

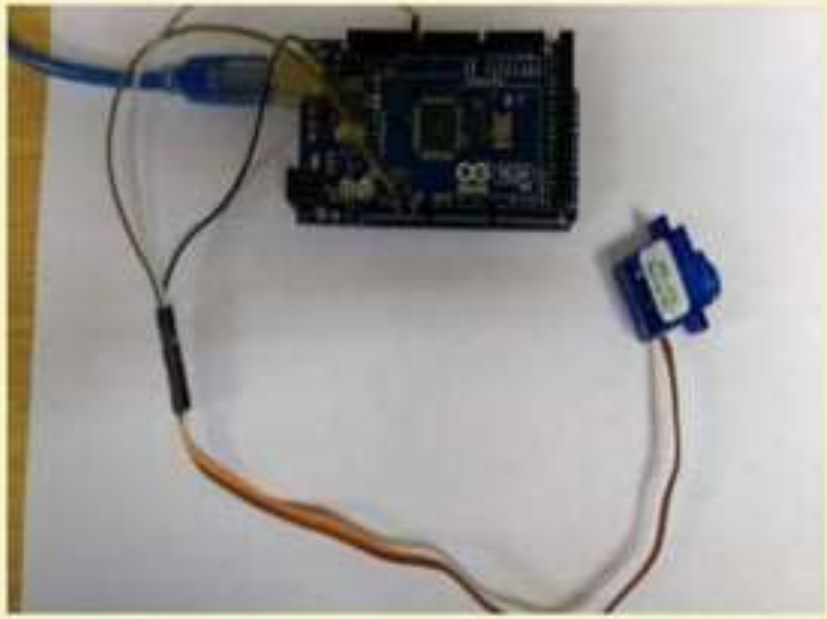
Sketch: SERVO_ACTUATOR (contd..)

- Create an instance of Servo
- The instance must be attached to the pin before being used in the code
- Write() function takes the degree value and rotates the motor accordingly

Sensor & Actuators

Connection

- Connect the Ground of the servo to the ground of the Arduino board.
- Connect the power supply wire to the 5V pin of the board.
- Connect the signal wire to any digital output pin (we have used pin 8).



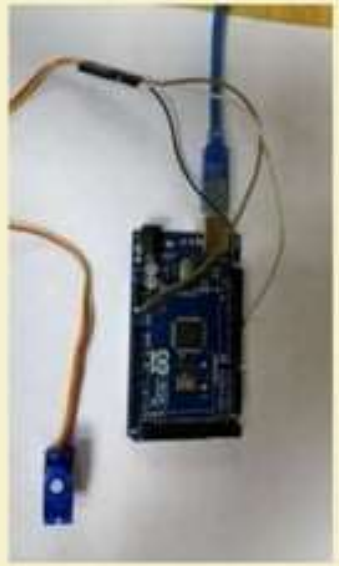
Sensor & Actuators

Board Setup

- Connect the board to the PC
- Set the port and board type
- Verify and upload the code

Output

The motor turns 0, 90 and 180 degrees with a delay of 1 second each.



Sensor & Actuators

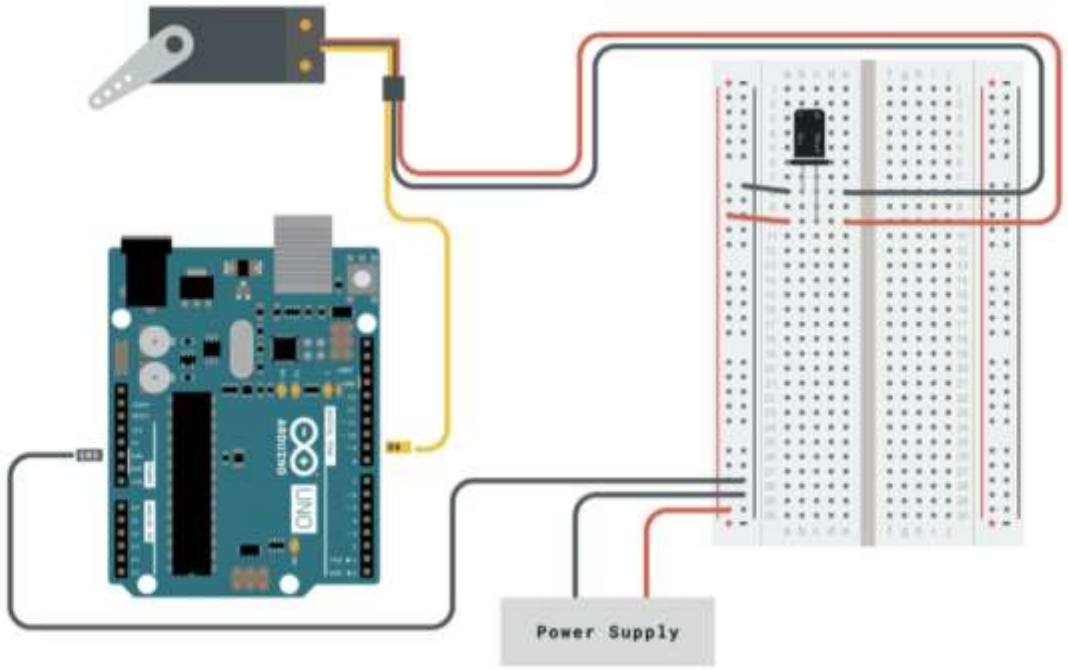
Do more with the Servo library

Some other functions available with the Servo library:

- Knob()
- Sweep()
- write()
- writeMicroseconds()
- read()
- attached()
- detach()

Sensor & Actuators

KNOB CIRCUIT



The Knob Circuit.

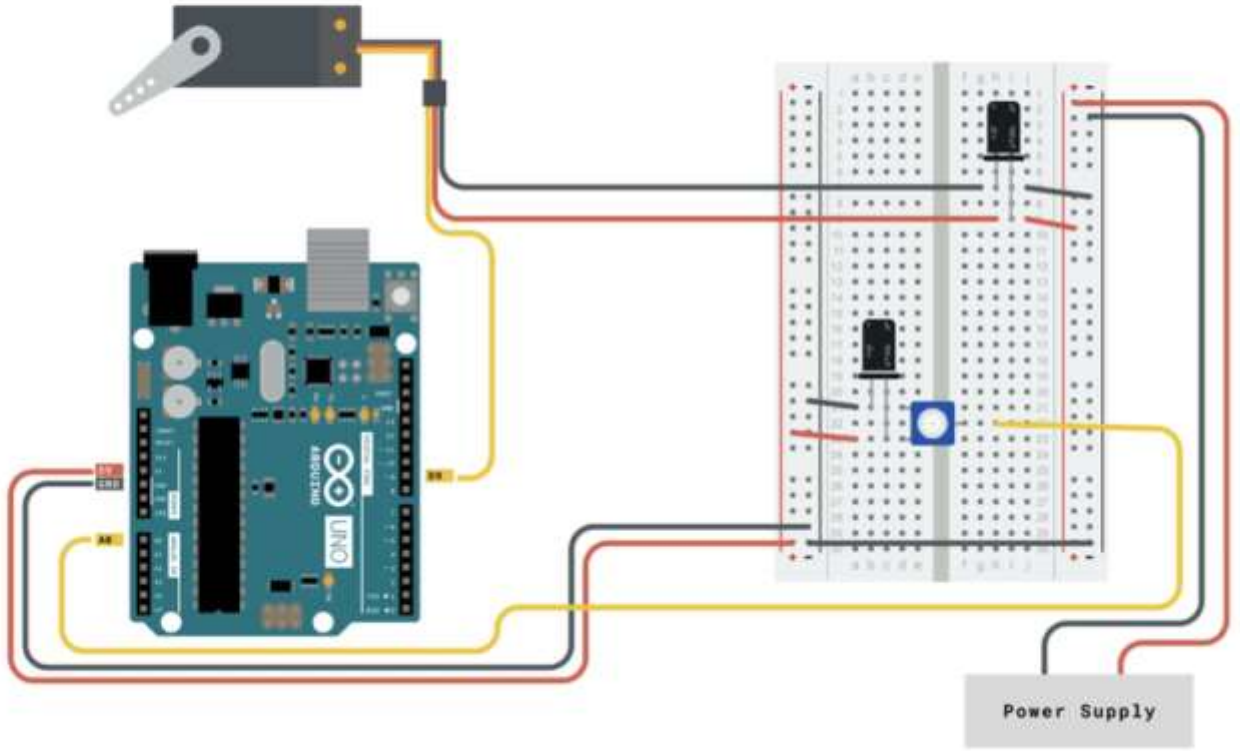
Sensor & Actuators

```
1  #include <Servo.h>
2
3  Servo myservo;  // create servo object to control a servo
4
5  int potpin = 0;  // analog pin used to connect the potentiometer
6  int val;    // variable to read the value from the analog pin
7
8  void setup() {
9    myservo.attach(9);  // attaches the servo on pin 9 to the servo object
10 }
11
12 void loop() {
13   val = analogRead(potpin);            // reads the value of the potentiometer
14   val = map(val, 0, 1023, 0, 180);     // scale it to use it with the servo (va
15   myservo.write(val);                  // sets the servo position according to
16   delay(15);                           // waits for the servo to get there
17 }
```

Sensor & Actuators

Sweep Circuit

For the **Sweep** example, connect the servo motor as shown in the circuit below.

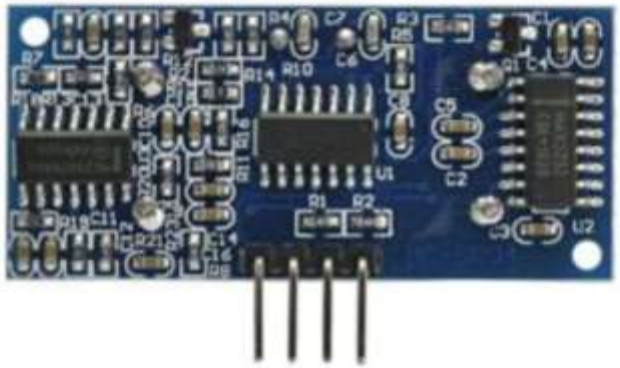


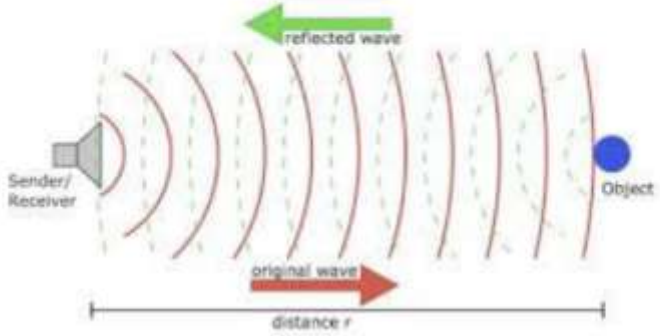
The Sweep Circuit.

Sensor & Actuators

```
#include <Servo.h>

Servo myservo;  // create servo object to control a servo
// twelve servo objects can be created on most boards

int pos = 0;    // variable to store the servo position

void setup() {
  myservo.attach(9);  // attaches the servo on pin 9 to the servo object
}

void loop() {
  for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees
    // in steps of 1 degree
    myservo.write(pos);              // tell servo to go to position in variabl
    delay(15);                       // waits 15ms for the servo to reach the p
  }
  for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees
    myservo.write(pos);              // tell servo to go to position in variabl
    delay(15);                       // waits 15ms for the servo to reach the p
  }
}
```

Sensor & Actuators

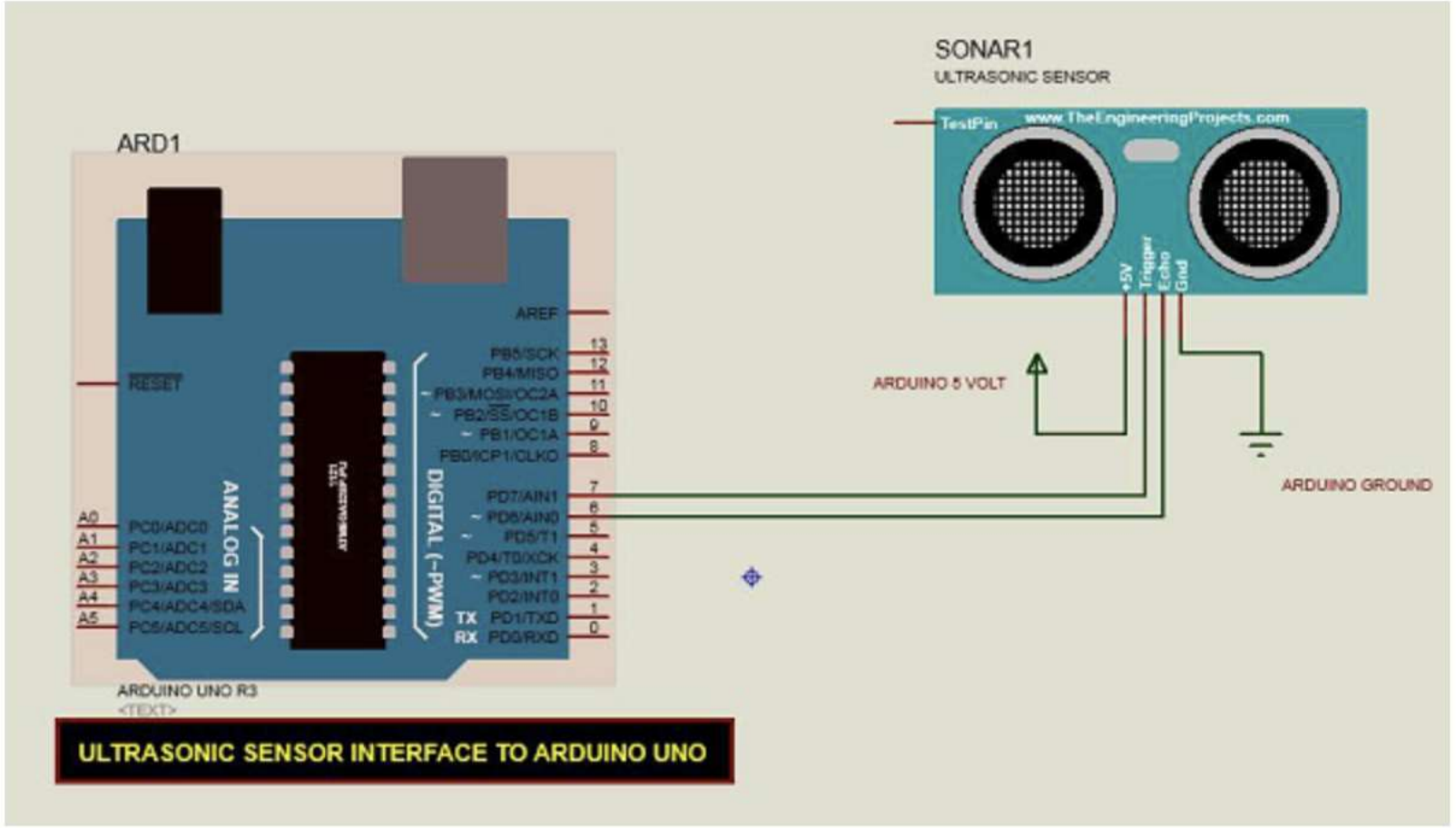




Technical Specifications

- Power Supply – +5V DC
- Quiescent Current – <2mA
- Working Current – 15mA
- Effectual Angle – <15°
- Ranging Distance – 2cm – 400 cm/1" – 13ft
- Resolution – 0.3 cm
- Measuring Angle – 30 degree

Sensor & Actuators



Sensor & Actuators

```
const int echoPin = 6; // Echo Pin of Ultrasonic Sensor

void setup() {
   Serial.begin(9600); // Starting Serial Terminal
}

void loop() {
   long duration, inches, cm;
   pinMode(pingPin, OUTPUT);
   digitalWrite(pingPin, LOW);
   delayMicroseconds(2);
   digitalWrite(pingPin, HIGH);
   delayMicroseconds(10);
   digitalWrite(pingPin, LOW);
   pinMode(echoPin, INPUT);
   duration = pulseIn(echoPin, HIGH);
   inches = microsecondsToInches(duration);
   cm = microsecondsToCentimeters(duration);
   Serial.print(inches);
   Serial.print("in, ");
   Serial.print(cm);
   Serial.print("cm");
   Serial.println();
   delay(100);
}

long microsecondsToInches(long microseconds) {
   return microseconds / 74 / 2;
}

long microsecondsToCentimeters(long microseconds) {
   return microseconds / 29 / 2;
}
```

MEMORY INTERFACE

EEPROM interface using SPI

- Arduino interface with an AT25HP512 Atmel serial EEPROM using the Serial Peripheral Interface (SPI) protocol.
- EEPROM chips are very useful for data storage.
- Note that the chip on the Arduino board contains an internal EEPROM.

EEPROM interface using SPI

- Serial Peripheral Interface (SPI) is a synchronous serial data protocol used by Microcontrollers for communicating with one or more peripheral devices quickly over short distances.

- With an SPI connection there is always one master device (usually a microcontroller) which controls the peripheral devices.

- Typically, there are three lines common to all the devices,
  - Master In Slave Out (MISO) - The Slave line for sending data to the master,
  - Master Out Slave In (MOSI) - The Master line for sending data to the peripherals,
  - Serial Clock (SCK) - The clock pulses which synchronize data transmission generated by the master, and
  - Slave Select pin - allocated on each device which the master can use to enable and disable specific devices and avoid false transmissions due to line noise.

EEPROM interface using SPI

- All SPI settings are determined by the Arduino SPI Control Register (SPCR).
- Control registers code control settings for various microcontroller functionalities. Usually, each bit in a control register effects a particular setting, such as speed or polarity.
- Data registers simply hold bytes. For example, the SPI data register (SPDR) holds the byte which is about to be shifted out the MOSI line, and the data which has just been shifted in the MISO line.
- Status registers change their state based on various microcontroller conditions. For example, the seventh bit of the SPI status register (SPSR) gets set to 1 when a value is shifted in or out of the SPI.

EEPROM interface using SPI

```
SPCR
| 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| SPIE | SPE  | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

SPIE - Enables the SPI interrupt when 1

SPE - Enables the SPI when 1

DORD - Sends data least Significant Bit First when 1, most Significant Bit first wh

MSTR - Sets the Arduino in master mode when 1, slave mode when 0

CPOL - Sets the data clock to be idle when high if set to 1, idle when low if set to

CPHA - Samples data on the falling edge of the data clock when 1, rising edge when

SPR1 and SPR0 - Sets the SPI speed, 00 is fastest (4MHz) 11 is slowest (250KHz)
```

EEPROM interface using SPI

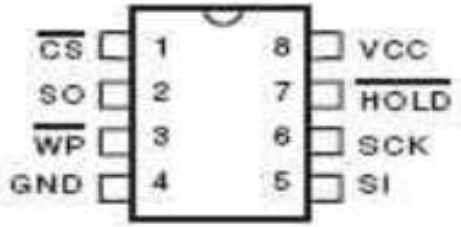
- AT25HP512 is a 65,536 byte serial EEPROM.
- It supports SPI modes 0 and 3, runs at up to 10MHz at 5v and can run at slower speeds down to 1.8v.
- It's memory is organized as 512 pages of 128 bytes each.
- It can only be written 128 bytes at a time, but it can be read 1-128 bytes at a time.
- The device also offers various degrees of write protection and a hold pin

EEPROM interface using SPI

Pin Configurations

Pin Name	Function
$\overline{CS}$	Chip Select
SCK	Serial Data Clock
SI	Serial Data Input
SO	Serial Data Output
GND	Ground
VCC	Power Supply
$\overline{WP}$	Write Protect
$\overline{HOLD}$	Suspends Serial Input

8-pin PDIP



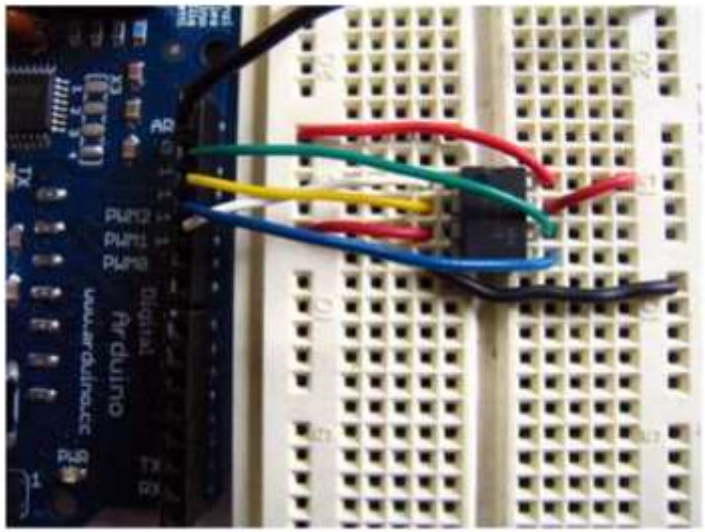
EEPROM interface using SPI

- The device is enabled by pulling the Chip Select (CS) pin low.
- Instructions are sent as 8 bit operational codes (opcodes) and are shifted in on the rising edge of the data clock.
- It takes the EEPROM about 10 milliseconds to write a page (128 bytes) of data, so a 10ms pause should follow each EEPROM write routine.

EEPROM interface using SPI

- Connect EEPROM pins 3, 7 and 8 to 5v and pin 4 to ground.

- Connect EEPROM pin 1 to Arduino pin 10 (Slave Select - SS), EEPROM pin 2 to Arduino pin 12 (Master In Slave Out - MISO), EEPROM pin 5 to Arduino pin 11 (Master Out Slave In - MOSI), and EEPROM pin 6 to Arduino pin 13 (Serial Clock - SCK).



EEPROM interface using SPI

- The first step is setting up our pre-processor directives.
- We define the pins we will be using for our SPI connection, DATAOUT, DATAIN, SPICLOCK and SLAVESELECT. Then we define our opcodes for the EEPROM.
- OPCODES:
  - WREN: Write Enable
  - WRDI: Write disable
  - RDSR: Read Status Register
  - WRSR: Write Status Register
  - READ: Read from EEPROM
  - WRITE: Write to EEPROM after WREN

```
#define DATAOUT 11//MOSI
#define DATAIN  12//MISO
#define SPICLOCK  13//sck
#define SLAVESELECT 10//ss

//opcodes
#define WREN  6
#define WRDI  4
#define RDSR  5
#define WRSR  1
#define READ  3
#define WRITE 2
```

EEPROM interface using SPI

- **char buffer [128]** this is a 128 byte array we will be using to store the data for the EEPROM write:

```
byte eeprom_output_data;
byte eeprom_input_data=0;
byte clr;
int address=0;
//data buffer
char buffer [128];
```

EEPROM interface using SPI

- First we initialize our serial connection, set our input and output pin modes and set the SLAVESELECT line high to start.
- This deselects the device and avoids any false transmission messages due to line noise:

```
void setup()
{

  Serial.begin(9600);

  pinMode(DATAOUT, OUTPUT);

  pinMode(DATAIN, INPUT);

  pinMode(SPICLOCK,OUTPUT);

  pinMode(SLAVESELECT,OUTPUT);

  digitalWrite(SLAVESELECT,HIGH); //disable device
```

EEPROM interface using SPI

- Now we set the SPI Control register (SPCR) to the binary value 01010000. In the control register each bit sets a different functionality.

- The eighth bit disables the SPI interrupt, the seventh bit enables the SPI, the sixth bit chooses transmission with the most significant bit going first, the fifth bit puts the Arduino in Master mode, the fourth bit sets the data clock idle when it is low, the third bit sets the SPI to sample data on the rising edge of the data clock, and the second and first bits set the speed of the SPI to system speed / 4 = 16 MHz / 4 = 4MHz (the fastest).
  If last 2 bits are 11, speed is slowest: 250KHz.

- After setting our control register up we read the SPI status register (SPSR) and data register (SPDR) in to the junk clr variable to clear out any spurious data from past runs:

EEPROM interface using SPI

```
// SPCR = 01010000

   //interrupt disabled,spi enabled,msb 1st,master,clk low when idle,

   //sample on leading edge of clk,system clock/4 rate (fastest)

   SPCR = (1<<SPE)|(1<<MSTR);

   clr=SPSR;

   clr=SPDR;

   delay(10);
```

EEPROM interface using SPI

- The EEPROM MUST be write enabled before every write instruction.
- To send the instruction we pull the SLAVESELECT line low, enabling the device, and then send the instruction using the spi_transfer function.
- Note that we use the WREN opcode we defined at the beginning of the program. Finally, we pull the SLAVESELECT line high again to release it:

```
//fill buffer with data

  fill_buffer();

  //fill eeprom w/ buffer

  digitalWrite(SLAVESELECT,LOW);

  spi_transfer(WREN);  //write enable

  digitalWrite(SLAVESELECT,HIGH);
```

EEPROM interface using SPI

- Now we pull the SLAVESELECT line low to select the device again after a brief delay.
- We send a WRITE instruction to tell the EEPROM we will be sending data to record into memory.
- We send the 16 bit address to begin writing at in two bytes, Most Significant Bit first.
- Next we send our 128 bytes of data from our buffer array, one byte after another without pause.
- Finally we set the SLAVESELECT pin high to release the device and pause to allow the EEPROM to write the data:

EEPROM interface using SPI

```
delay(10);

  digitalWrite(SLAVESELECT,LOW);

  spi_transfer(WRITE);  //write instruction

  address=0;

  spi_transfer((char)(address>>8));     //send MSByte address first

  spi_transfer((char)(address));        //send LSByte address

  //write 128 bytes

  for  (int  I=0;I<128;I++)

  {

     spi_transfer(buffer[I]);  //write data byte

  }

  digitalWrite(SLAVESELECT,HIGH);  //release chip

  //wait for eeprom to finish writing

  delay(3000);
```

EEPROM interface using SPI

- In the main loop we just read one byte at a time from the EEPROM and print it out the serial port. We add a line feed and a pause for readability.
- Each time through the loop we increment the EEPROM address to read.
- When the address increments to 128 we turn it back to 0 because we have only filled 128 addresses in the EEPROM with data.

```
void loop()
{

  eeprom_output_data = read_eeprom(address);

  Serial.print(eeprom_output_data,DEC);

  Serial.print('\n',BYTE);

  address++;

  delay(500);  //pause for readability
}
```

EEPROM interface using SPI

- The spi_transfer function loads the output data into the data transmission register, thus starting the SPI transmission.
- It polls a bit to the SPI Status register (SPSR) to detect when the transmission is complete using a bit mask, SPIF.
- It then returns any data that has been shifted in to the data register by the EEPROM.

```
char spi_transfer(volatile char data)
{

  SPDR = data;                          // Start the transmission

  while (!(SPSR & (1<<SPIF)))       // Wait for the end of the transmission

  {

  };

  return SPDR;                          // return the received byte
}
```

EEPROM interface using SPI

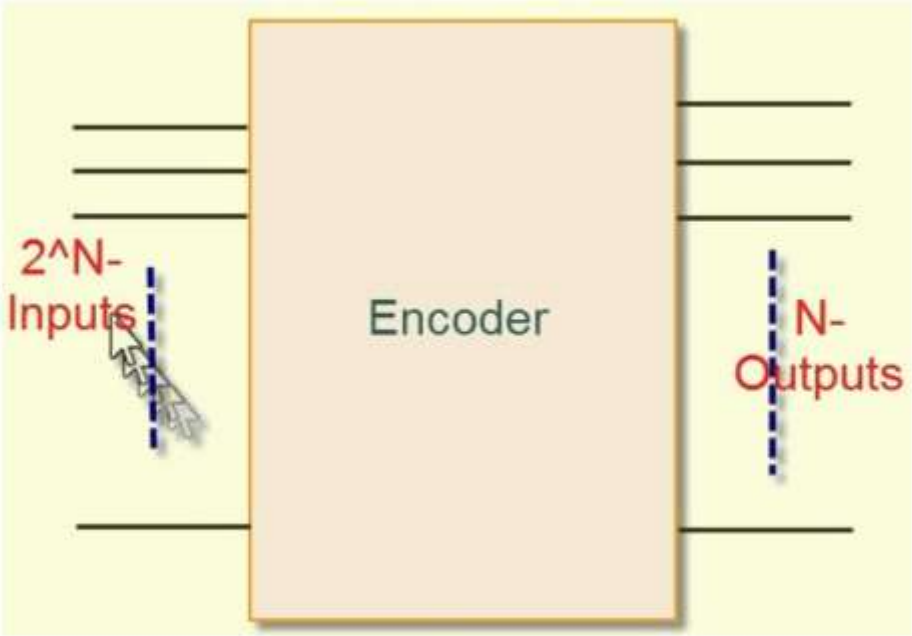
- The read_eeprom function allows us to read data back out of the EEPROM.
- First we set the SLAVESELECT line low to enable the device.
- Then we transmit a READ instruction, followed by the 16-bit address we wish to read from, Most Significant Bit first.
- Next we send a dummy byte to the EEPROM for the purpose of EEPROM shifting the next data out on MISO line.
- Finally we pull the SLAVESELECT line high again to release the device after reading one byte, and return the data.

EEPROM interface using SPI

```
byte read_eeprom(int EEPROM_address)
{

  //READ EEPROM

  int data;

  digitalWrite(SLAVESELECT,LOW);

  spi_transfer(READ);  //transmit read opcode

  spi_transfer((char)(EEPROM_address>>8));     //send MSByte address first

  spi_transfer((char)(EEPROM_address));        //send LSByte address

  data = spi_transfer(0xFF);  //get data byte

  digitalWrite(SLAVESELECT,HIGH);  //release chip, signal end transfer

  return data;
}
```

Encoders & Decoders

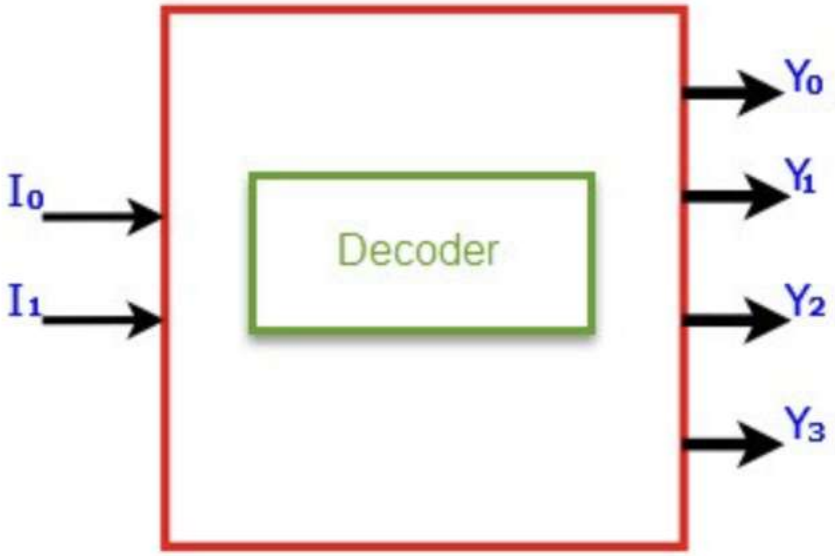
ENCODER



Encoder

Encoders & Decoders

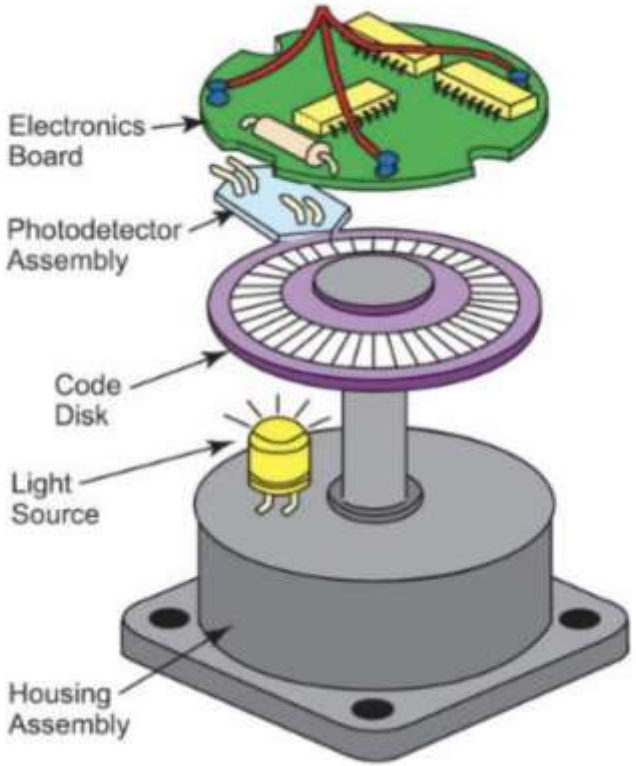
DECODER



Decoder

Encoders & Decoders

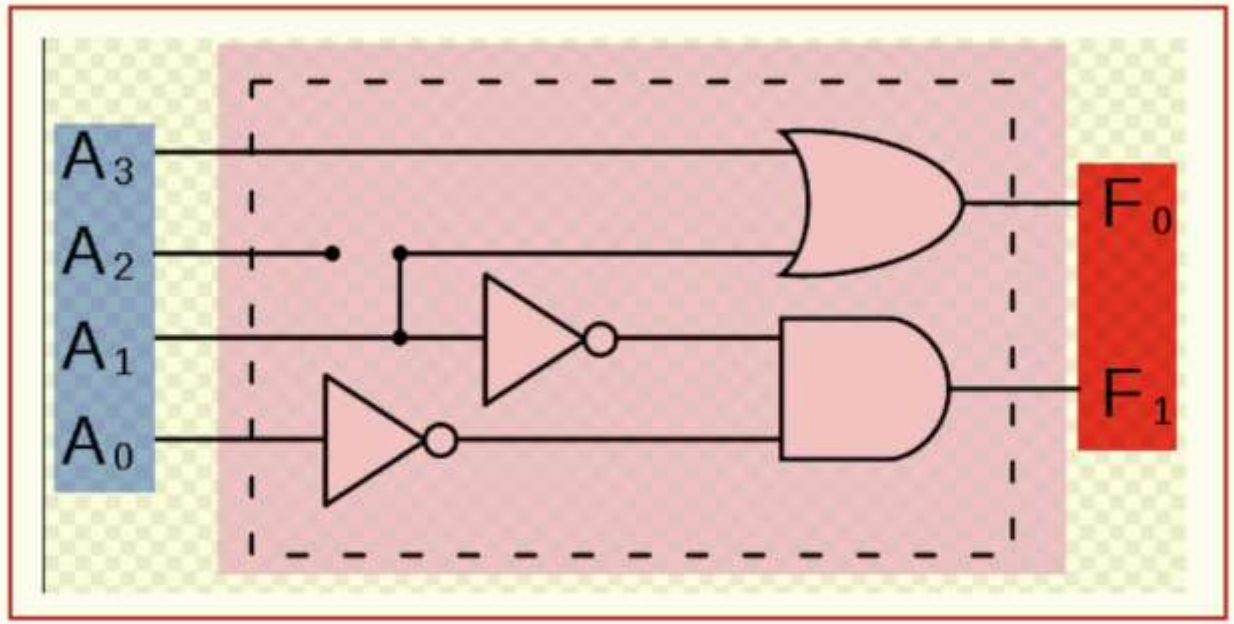
WORKING OF ENCODER



Working of Encoder

Encoders & Decoders

LOGICS AND TRUTH TABLE



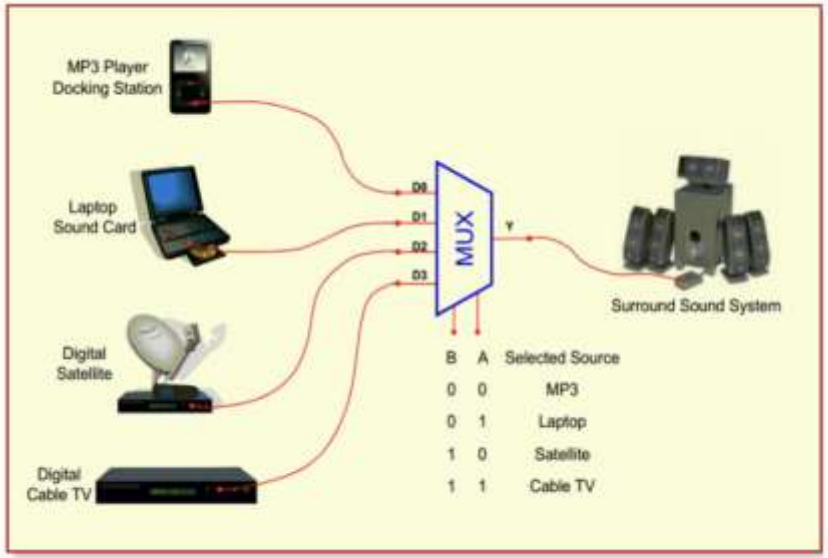
Simple Encoder

I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

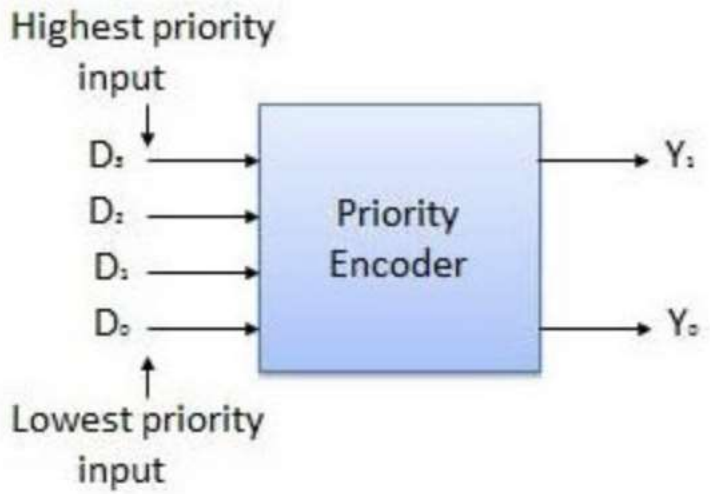
Encoder Truth Table

Encoders & Decoders

TYPES OF ENCODERS



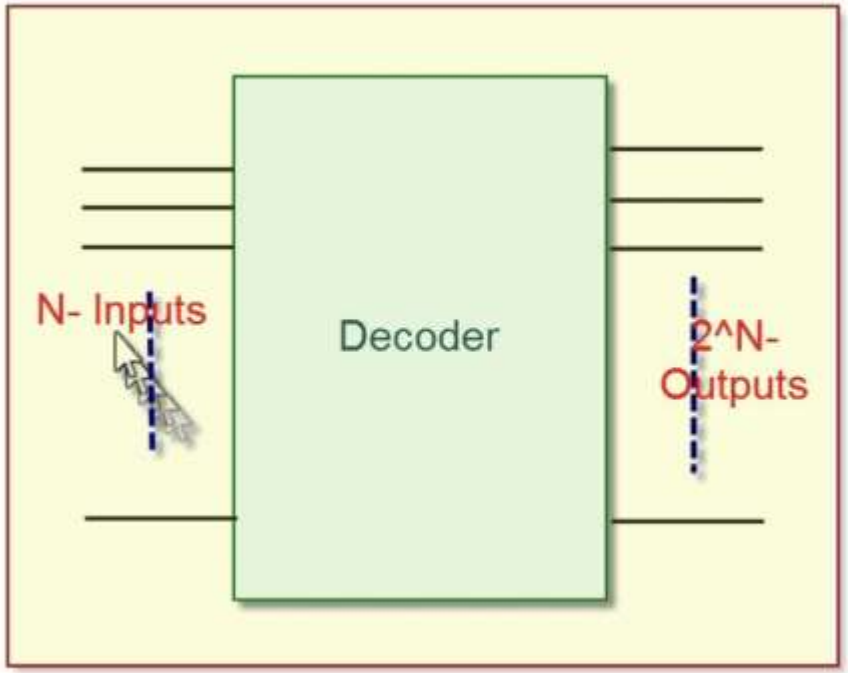
Multiplexer



Priority Encoder

Encoders & Decoders

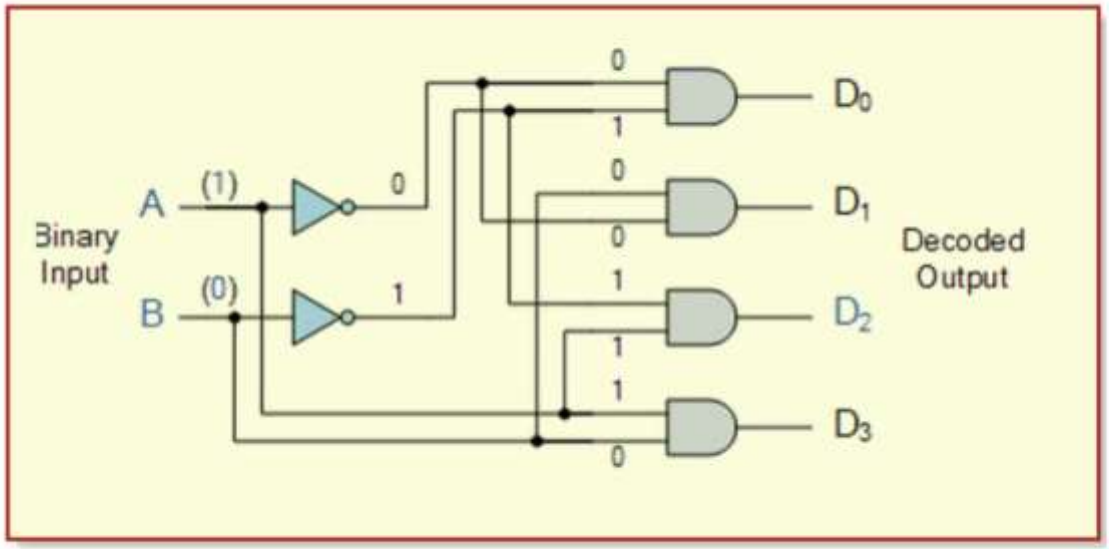
DECODERS



Decoder

Encoders & Decoders

LOGICS AND TRUTH TABLE



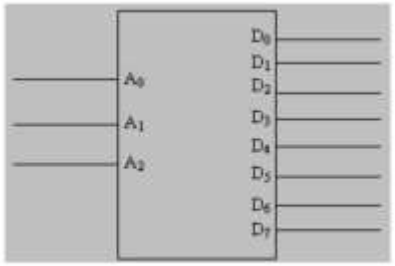
Decoder Circuit

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Decoder Truth Table

Encoders & Decoders

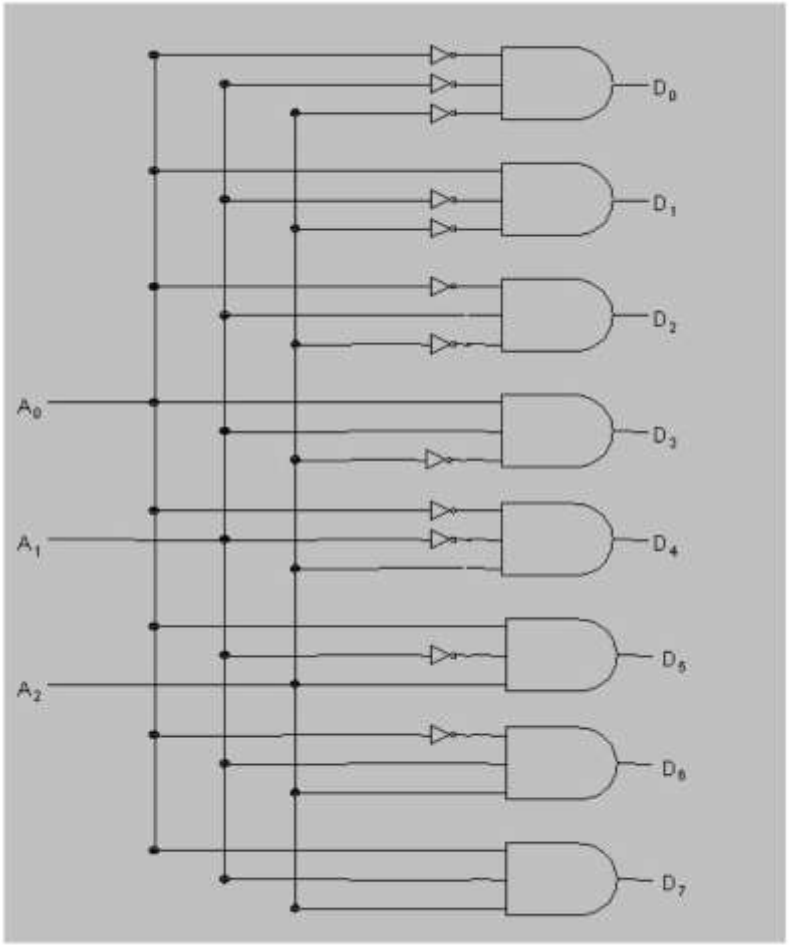
3 TO 8 DECODER



3 to 8 Decoder

Decimal Digit	Binary Inputs			Logic Function	Outputs							
					D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	0	$\overline{A_2}\,\overline{A_1}\,\overline{A_0}$	1	0	0	0	0	0	0	0
1	0	0	1	$\overline{A_2}\,\overline{A_1}\,A_0$	0	1	0	0	0	0	0	0
2	0	1	0	$\overline{A_2}\,A_1\,\overline{A_0}$	0	0	1	0	0	0	0	0
3	0	1	1	$\overline{A_2}\,A_1\,A_0$	0	0	0	1	0	0	0	0
4	1	0	0	$A_2\,\overline{A_1}\,\overline{A_0}$	0	0	0	0	1	0	0	0
5	1	0	1	$A_2\,\overline{A_1}\,A_0$	0	0	0	0	0	1	0	0
6	1	1	0	$A_2\,A_1\,\overline{A_0}$	0	0	0	0	0	0	1	0
7	1	1	1	$A_2\,A_1\,A_0$	0	0	0	0	0	0	0	1

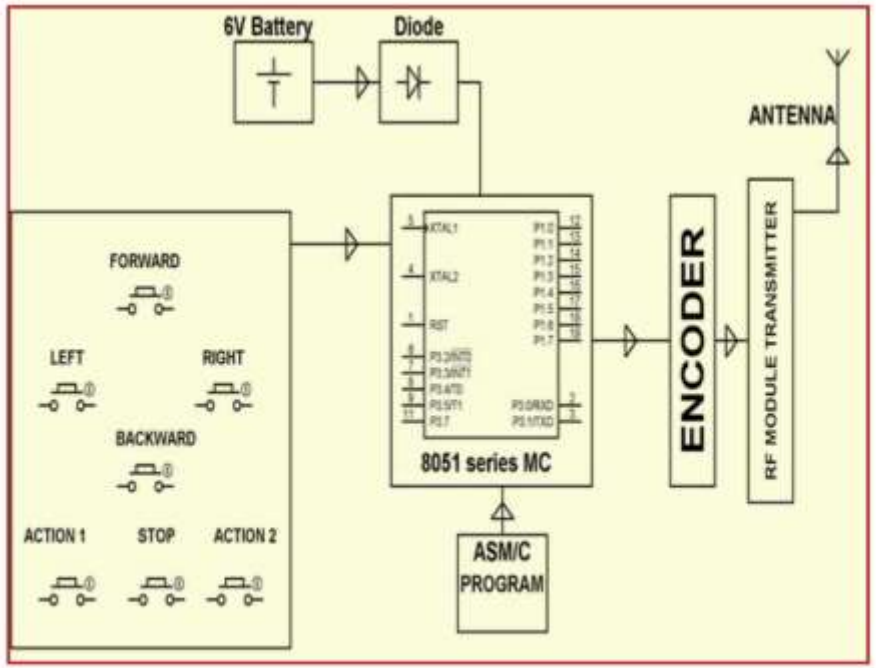
Truth Tabl o 3-to-8-Decoder



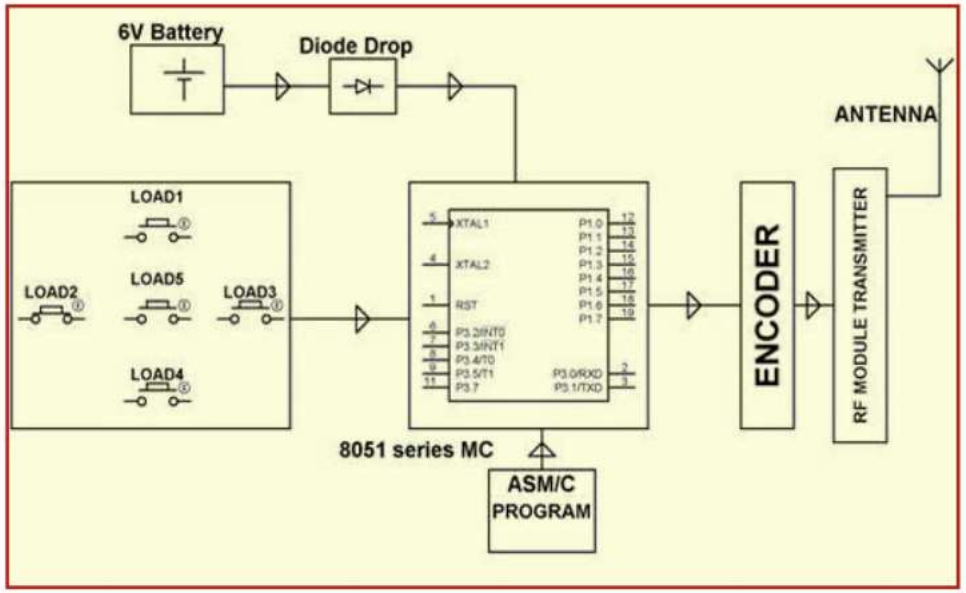
Logic Circuit of 3-to-8 Decoder

Encoders & Decoders

APPLICATIONS



War Field Flying Robort with NJight Vision Flying Camera



Robotic Vehicle with Metal Detector

Watch Dog Timers

WATCH DOG TIMERS

The Arduino UNO board has ATmega328P chip as its controlling unit.

The ATmega328P has a Watchdog Timer which is a useful feature to help the system recover from scenarios where the system hangs or freezes due to errors in the code written or due to conditions that may arise due to hardware issues.

Watch Dog Timers

WATCH DOG TIMERS

How does the Watchdog timer work?

The watchdog timer needs to be configured according to the need of the application.

The watchdog timer uses an internal 128kHz clock source.

When enabled, it starts counting from 0 to a value selected by the user. If the watchdog timer is not reset by the time it reaches the user selected value, the watchdog resets the microcontroller.

The ATmega328P watchdog timer can be configured for 10 different time settings (the time after which the watchdog timer overflows, thus causing a reset).

The various times are : 16ms, 32ms, 64ms, 0.125s, 0.25s, 0.5s, 1s, 2s, 4s and 8s.

Watch Dog Timers

WATCH DOG TIMERS

Example

Let's see how to configure the watchdog timer for an Arduino UNO board.

Here, we will use a simple example of LED blinking.

The LEDs are blinked for a certain time before entering a while(1) loop. The while(1) loop is used as a substitute for a system in the hanged state.

Since the watchdog timer is not reset when in the while(1) loop, the watchdog causes a system reset and the LEDs start blinking again before the system hangs and restarts again. This continues in a loop.

Here, we will be using the on-board LED connected to the pin 13 of the Arduino UNO board. For this example sketch, the only thing required is the Arduino UNO board.

Watch Dog Timers

WATCH DOG TIMERS

```
#include<avr/wdt.h> /* Header for watchdog timers in AVR */

void setup() {
  Serial.begin(9600); /* Define baud rate for serial communication */
  Serial.println("Watchdog Demo Starting");
  pinMode(13, OUTPUT);
  wdt_disable();  /* Disable the watchdog and wait for more than 2 seconds
  delay(3000);  /* Done so that the Arduino doesn't keep resetting infinitely ir
  wdt_enable(WDTO_2S);  /* Enable the watchdog with a timeout of 2 second
}

void loop() {
  for(int i = 0; i<20; i++) /* Blink LED for some time */
  {
    digitalWrite(13, HIGH);
    delay(100);
    digitalWrite(13, LOW);
    delay(100);
    wdt_reset();  /* Reset the watchdog */
  }
  while(1); /* Infinite loop. Will cause watchdog timeout and system reset. */
}
```

THANKS